

The Manifold Adventure, revision r5

Walkthrough of every level with its solution, and a report of the update and its performance.

Generated 2026-09-06 from `mathlib-manifolds-de3a9cf330-r5` on Mathlib `de3a9cf330`, Lean fork `cauli/lean4@62b6a22913` (upstream `ecf55de08b`).

10 worlds, 45 levels, 45 kernel-verified reference solutions. Cold start to a provable first level: 491.9 s. Median kernel check per level once warm: 29 ms.

Contents

1. What changed from r4 to r5
2. Course map
3. Performance walkthrough
4. One trail, one leaf (4 levels)
5. One pole is missing (5 levels)
6. A stack of maps (5 levels)
7. Smooth manifolds (5 levels)
8. A direction at every point (5 levels)
9. Identity and product charts (5 levels)
10. How smooth is smooth (3 levels)
11. The dial comes around (5 levels)
12. Two hinges, one reach (4 levels)
13. Can the arm touch it? (4 levels)

What changed from r4 to r5

Revision r4 had 9 worlds and 40 levels. Most of the main path proved bookkeeping facts about charts by citing a Mathlib lemma, while the ideas that make a manifold a manifold, the sphere that one chart cannot cover, smooth transition maps, and tangent vectors as velocities, had one thin level each or none. Revision r5 rebuilds the main path around those ideas and moves the bookkeeping to optional worlds or to inventory grants.

Design rules, each enforced by a unit test

- The sphere comes second, before any general vocabulary. Every new word is checked on the sphere before it is stated in general.
- Transition maps are stated in general as groupoid membership, unpacked to `ContDiffOn`, and checked on the sphere.
- Tangent vectors are shown to be genuine vectors of the surrounding space, and the tangent plane is proved to be the orthogonal complement of the radius.
- No main-path level is closed by `infer_instance`.
- Key levels carry a "Think first" prompt with a hidden answer, and no prompt names a tactic.
- Level ids are unchanged, so saved progress and the r4 conformance ids carry over.

Ten new levels, all in one new Lean module `ManifoldAdventure.Course`

1. **One of the two leaves shows her place** (`two_leaves_cover_point`), in One pole is missing
2. **The preferred leaf comes from the far pole** (`sphere_preferred_chart`), in A stack of maps
3. **Every projection is filed in the atlas** (`sphere_chart_in_atlas`), in A stack of maps
4. **Changing leaves is a smooth move** (`chart_change_is_smooth`), in Smooth manifolds
5. **The smooth move, spelled out in calculus** (`chart_change_contDiffOn`), in Smooth manifolds
6. **Two drawings of the bead agree smoothly** (`sphere_chart_change_smooth`), in Smooth manifolds
7. **Leaving the bead for the room around it** (`sphere_inclusion_smooth`), in A direction at every point
8. **Her velocities are real vectors** (`tangent_vectors_are_vectors`), in A direction at every point
9. **The tangent plane stands square to the radius** (`tangent_plane_is_orthogonal`), in A direction at every point
10. **Two angles, one pointer position** (`same_point_iff_full_turns`), in The dial comes around

Levels that left the main path

- Into the route book (homeomorph composition): its lemma is now granted on level 1.
- This leaf is in the atlas, The map works nearby: granted on the first atlas level.
- Every mark has a place on the bead (surjectivity): granted on the first sphere level.
- Identity and product charts (5 levels): now the optional world "Identity and product charts".
- The model space is a manifold, Passing an easier check, The smooth atlas passes the basic check: now the optional world "How smooth is smooth".
- Standing still anywhere (zero bundle point): removed; its remark lives in the first tangent level.

Interactive labs

- **Projection lab** (SVG): a draggable point on a circle, both stereographic poles, the dashed rays to the leaf, and a readout of the two readings and their product. Uses Mathlib's normalisation, so the product is 4 and the transition map is $x \mapsto 4/x$. On the sphere world page and six levels.
- **Angle lab** (SVG): a dial and the angle line, two arc charts with their missing points, and the readings' difference on the overlap. On the atlas world page and two circle levels.

- **Tangent lab** (three.js): a sphere with a draggable point, the tangent plane, the radius, and a velocity arrow with its dot product against the radius. On the tangent world page and three levels.

Build and verification

- The ten r4 Lean modules regenerate byte-identical, so the deployed Mathlib layers are unchanged. The new module imports the three leaves of the old module graph and adds no Mathlib imports.
- The new module was compiled with the wasm Lean under Node (no native Mathlib on the build machine); its olean carries the browser compiler's githash and ships as an eleventh layer of 28 KB.
- Conformance record mathlib-manifolds-de3a9cf330-r5: 45/45 reference solutions pass on the wasm kernel with the inventory policy, plus the partial goal preview and two policy rejections. Recorded 2026-09-06T10:32:10.034Z.

Course map

#	World	Levels	Path	Requires
1	One trail, one leaf	4	main	none
2	One pole is missing	5	main	Charts
3	A stack of maps	5	main	Sphere
4	Smooth manifolds	5	main	ChartedSpaces
5	A direction at every point	5	main	SmoothManifolds
6	Identity and product charts	5	optional	ChartedSpaces
7	How smooth is smooth	3	optional	SmoothManifolds
8	The dial comes around	5	optional	SmoothManifolds
9	Two hinges, one reach	4	optional	CircleMotion
10	Can the arm touch it?	4	optional	RobotArm

Main path: 24 levels in 5 worlds. Optional: 21 levels in 5 worlds.

Performance walkthrough

One headless Chromium session (Playwright, local dev server, assets already in the browser cache) played the course in order: a fresh page load on the first level, then in-app navigation to every world page and every level, pasting each reference solution into the editor, waiting for the live goal preview, and pressing Verify. Recorded 2026-09-06T12:19:03.724Z to 2026-09-06T12:29:16.275Z.

Cold start on the first level

Page heading visible after 417 ms. Lean ready (Verify enabled) after **491.9 s**. Phases as reported by the runtime widget:

At	Runtime status
433 ms	Starting the local Lean runtime...
2.5 s	Loading Lean environment: 183 / 230 MB
3.0 s	Loading Mathlib homeomorphisms: pack 1 / 37

At	Runtime status
3.5 s	Loading Mathlib homeomorphisms: pack 14 / 37
4.1 s	Loading Mathlib homeomorphisms: pack 25 / 37
4.6 s	Reading the local Mathlib local charts package...
5.1 s	Loading Mathlib smooth manifolds: pack 4 / 10
5.6 s	Loading Mathlib map projections: pack 4 / 22
6.1 s	Loading Mathlib map projections: pack 16 / 22
6.6 s	Loading Mathlib robot arm: pack 1 / 1
7.2 s	Opening the course in Lean...
8.2 s	Importing Lean modules: N / 4833
15.7 s	Importing Lean modules: N / 4833Still working
491.9 s	(ready)

Since every level shares one Lean environment, this import is paid once per browser session. World and level changes afterwards do not reload Mathlib.

Warm pages, all 45 levels

Measure	Min	Median	Max
Navigation to a level (click to heading)	27 ms	39 ms	805 ms
Lean ready on the level (Verify enabled)	1 ms	2 ms	208 ms
Live goal preview of the solution	790 ms	795 ms	6.8 s
Verify round trip (click to result)	58 ms	112 ms	336 ms
Kernel elaboration and checking (reported by Lean)	16 ms	29 ms	255 ms

45 of 45 reference solutions were accepted by the kernel during the walkthrough.

Every page

Page	Nav	Ready	Preview	Verify	Kernel	Result
World: One trail, one leaf	60 ms					lab page
homeomorphisms-1 The drawing matches the trail	49 ms	2 ms	6.8 s	74 ms	19 ms	accepted
localcharts-3 Her mark lands in the drawing	41 ms	3 ms	798 ms	69 ms	21 ms	accepted
localcharts-4 Back to the same spot	35 ms	1 ms	827 ms	115 ms	26 ms	accepted
localcharts-5 The leaf reads back into the patch	67 ms	1 ms	795 ms	111 ms	28 ms	accepted
World: One pole is missing	26 ms					lab page
mapprojections-1 The pole stays off the leaf	39 ms	3 ms	1.8 s	112 ms	52 ms	accepted
mapprojections-2 The far pole lands in the middle	34 ms	2 ms	794 ms	214 ms	85 ms	accepted
mapprojections-4 Off the pole, onto the leaf	34 ms	1 ms	791 ms	114 ms	58 ms	accepted
course-1 One of the two leaves shows her place	39 ms	1 ms	794 ms	227 ms	79 ms	accepted
mapprojections-5 The second leaf covers the hole	33 ms	2 ms	797 ms	128 ms	70 ms	accepted

Page	Nav	Ready	Preview	Verify	Kernel	Result
World: A stack of maps	22 ms					lab page
chartedspaces-1 A leaf for where she stands	38 ms	8 ms	792 ms	63 ms	17 ms	accepted
chartedspaces-3 Her place lands on the leaf	36 ms	2 ms	790 ms	62 ms	20 ms	accepted
chartedspaces-5 No place left uncovered	70 ms	2 ms	1.8 s	95 ms	19 ms	accepted
course-2 The preferred leaf comes from the far pole	46 ms	2 ms	791 ms	210 ms	79 ms	accepted
course-3 Every projection is filed in the atlas	31 ms	1 ms	791 ms	107 ms	56 ms	accepted
World: Smooth manifolds	29 ms					lab page
smoothmanifolds-1 Two leaves in conversation	30 ms	2 ms	792 ms	117 ms	27 ms	accepted
course-4 Changing leaves is a smooth move	39 ms	2 ms	1.3 s	112 ms	33 ms	accepted
course-5 The smooth move, spelled out in calculus	35 ms	3 ms	795 ms	111 ms	47 ms	accepted
course-6 Two drawings of the bead agree smoothly	38 ms	2 ms	3.3 s	316 ms	255 ms	accepted
smoothmanifolds-5 Two circles make a torus	37 ms	2 ms	792 ms	115 ms	43 ms	accepted
World: A direction at every point	219 ms					lab page
tangentspaces-1 Ada stands still	252 ms	16 ms	798 ms	102 ms	29 ms	accepted
tangentspaces-2 Read the location tag	261 ms	6 ms	809 ms	93 ms	27 ms	accepted
course-7 Leaving the bead for the room around it	688 ms	11 ms	1.3 s	228 ms	90 ms	accepted
course-8 Her velocities are real vectors	750 ms	70 ms	799 ms	232 ms	110 ms	accepted
course-9 The tangent plane stands square to the radius	805 ms	14 ms	1.8 s	336 ms	191 ms	accepted
World: Identity and product charts	19 ms					lab page
canonicalcharts-1 The reference grid stays put	75 ms	2 ms	795 ms	62 ms	16 ms	accepted
canonicalcharts-2 The identity is filed in the atlas	34 ms	1 ms	791 ms	58 ms	16 ms	accepted
canonicalcharts-3 Only the identity is filed there	35 ms	2 ms	791 ms	64 ms	18 ms	accepted
canonicalcharts-4 Two readings at once	248 ms	35 ms	847 ms	180 ms	30 ms	accepted
canonicalcharts-5 The paired chart contains her place	128 ms	2 ms	851 ms	179 ms	29 ms	accepted
World: How smooth is smooth	27 ms					lab page
smoothmanifolds-2 The reference leaf is ready	36 ms	2 ms	794 ms	114 ms	50 ms	accepted
smoothmanifolds-3 Passing an easier check	52 ms	1 ms	796 ms	113 ms	26 ms	accepted
smoothmanifolds-4 The smooth atlas passes the basic check	34 ms	1 ms	792 ms	113 ms	29 ms	accepted
World: The dial comes around	25 ms					lab page
circlemotion-1 No turn leaves the pointer home	27 ms	1 ms	794 ms	63 ms	18 ms	accepted
circlemotion-2 Two turns compose	33 ms	2 ms	792 ms	105 ms	28 ms	accepted
circlemotion-3 One full turn changes nothing	37 ms	1 ms	790 ms	67 ms	21 ms	accepted
course-10 Two angles, one pointer position	38 ms	2 ms	793 ms	80 ms	26 ms	accepted
circlemotion-4 The pointer turns smoothly	42 ms	4 ms	4.3 s	112 ms	51 ms	accepted
World: Two hinges, one reach	323 ms					lab page
robotarm-1 Find the tip of the arm	142 ms	208 ms	834 ms	202 ms	46 ms	accepted

Page	Nav	Ready	Preview	Verify	Kernel	Result
robotarm-2 Both bars point forward	91 ms	1 ms	792 ms	111 ms	39 ms	accepted
robotarm-3 A full shoulder turn reaches the same point	66 ms	1 ms	825 ms	100 ms	24 ms	accepted
robotarm-4 The arm moves without a jump	255 ms	22 ms	809 ms	152 ms	67 ms	accepted
World: Can the arm touch it?	25 ms					lab page
robotreachability-1 The arm has an outer limit	37 ms	2 ms	1.3 s	219 ms	78 ms	accepted
robotreachability-2 The folded arm leaves a gap	32 ms	2 ms	1.3 s	316 ms	187 ms	accepted
robotreachability-3 Every pose stays in the ring	31 ms	1 ms	793 ms	71 ms	22 ms	accepted
robotreachability-4 Outside the ring is out of reach	40 ms	9 ms	795 ms	111 ms	29 ms	accepted

Course introduction

The Manifold Adventure

Ada is an ant, so she can only inspect her world from the inside. Manifold theory takes the same point of view: understand the whole space through local coordinates.

The main path is short and concrete. It starts with one chart, moves at once to a sphere that no single chart can cover, then builds the general words: an [atlas](#) of charts, smooth changes between them ([IsManifold](#)), and the [TangentSpace](#) of velocities at a point. Every word is checked on the sphere before it is stated in general. Optional paths go deeper into Mathlib's chart instances, regularity orders, circular motion, and a two-joint robot arm whose configuration space is a torus.

About this course

This course was written for this local library. It uses the interface from [Lean4Game](#), primarily developed by Alexander Bentkamp and Jon Eugster. The mathematical sources used to write the lessons are credited and linked above.

One trail, one leaf

Manifold Adventure

WIP · Work in progress

Games

Worlds

Lean playground

Game source

Back to world tree

LOCAL KERNEL

One trail, one leaf

0/4 complete

One path, two descriptions

Ada begins on a single trail. She can copy the whole route onto one leaf, matching every place on the trail with one place in the drawing. A `Homeomorph` is Mathlib's bundled version of such a correspondence: an equivalence together with continuity proofs in both directions.

The trail soon climbs onto a rounded stone. From where Ada stands she can survey only the patch around her, so she draws just the part she can see. Mathlib represents one local chart by `OpenPartialHomeomorph`: it has a `source` on the stone, a `target` in the drawing, and inverse laws that apply inside the patch.

Glossary for this course: a leaf is a chart, the shaded patch is its `source`, the drawing is its `target`, a stack of leaves is an atlas, and the stone or bead is the manifold. The first level names its instance assumptions, such as `trailTopology`, so the lesson can point at them. Later levels leave them anonymous, which is ordinary Lean style.

01

The drawing matches the trail

Local kernel

02

Her mark lands in the drawing

Local kernel

03

Back to the same spot

Local kernel

04

The leaf reads back into the patch

Local kernel

Convenience port of Lean4Game; upstream creators and licenses retain credit.

Mathlib proof checking stays in this browser.

One path, two descriptions

Ada begins on a single trail. She can copy the whole route onto one leaf, matching every place on the trail with one place in the drawing. A `Homeomorph` is Mathlib's bundled version of such a correspondence: an equivalence together with continuity proofs in both directions.

The trail soon climbs onto a rounded stone. From where Ada stands she can survey only the patch around her, so she draws just the part she can see. Mathlib represents one local chart by `OpenPartialHomeomorph`: it has a `source` on the stone, a `target` in the drawing, and inverse laws that apply inside the patch.

Glossary for this course: a leaf is a chart, the shaded patch is its `source`, the drawing is its `target`, a stack of leaves is an atlas, and the stone or bead is the manifold. The first level names its instance assumptions, such as `trailTopology`, so the lesson can point at them. Later levels leave them anonymous, which is ordinary Lean style.

Measured: world page opened in 60 ms.

1. The drawing matches the trail

One trail, one leaf · Level 1 of 4

The drawing matches the trail

Ada stands midway along a path. North leads back to the nest; south leads to a patch of berries. She copies the path onto a leaf. As she moves a little along the trail, her mark should move only a little on the drawing. There can be no sudden jump.

Here `Trail` is the actual path and `Drawing` is the line Ada drew. The objects `trailTopology` and `drawingTopology` tell Lean what it means for points to be nearby in each space. Then `trailMap : Trail  $\approx$  Drawing` matches their points homeomorphically. It already contains a proof that its forward function is continuous, exposed as `Homeomorph.continuous` or `trailMap.continuous`.

Objective: Prove that the map from the actual trail to Ada's drawing is continuous.

Active goal

Objects

- `Trail` : Type `u`
- `Drawing` : Type `v`
- `trailTopology` : TopologicalSpace `Trail` — Continuous `trailMap`
- `drawingTopology` : TopologicalSpace `Drawing`
- `trailMap` : `Trail  $\approx$  Drawing`

Your proof

Enter tactics only. The surrounding `by` is supplied for you.

by 1

Available tactics

- `apply` `by_cases` `calc` `constructor`
- `exact` `ext` `fun_ext` `have`
- `infer_instance` `intro` `left` `obtain`
- `rcases` `refl` `right` `rw` `simp`
- `simp` `unfold`

World tree

- One trail, one leaf 8/1
 - 1 The drawing matches the trail
 - 2 Her mark lands in the drawing
 - 3 Back to the same spot
 - 4 The leaf needs back into the patch
- One path is missing 8/1
- A stack of maps 8/1
- Smooth manifolds 8/1

Buttons: Verify answer, Show a hint, View solution

Ada stands midway along a path. North leads back to the nest; south leads to a patch of berries. She copies the path onto a leaf. As she moves a little along the trail, her mark should move only a little on the drawing. There can be no sudden jump.

Here `Trail` is the actual path and `Drawing` is the line Ada drew. The objects `trailTopology` and `drawingTopology` tell Lean what it means for points to be nearby in each space. Then `trailMap : Trail  $\approx$  Drawing` matches their points homeomorphically. It already contains a proof that its forward function is continuous, exposed as `Homeomorph.continuous` or `trailMap.continuous`.

Three more facts about `trailMap` ride along in the same bundle and join the inventory now: the reverse map `trailMap.symm` is continuous, `Homeomorph.continuous_symm`; the round trip `trailMap.symm (trailMap place) = place` is `Homeomorph.symm_apply_apply`; and composing two such maps applies them in order, `Homeomorph.trans_apply`.

Objective: Prove that the map from the actual trail to Ada's drawing is continuous.

GOAL

```
theorem homeomorph_continuous {Trail : Type u} {Drawing : Type v}
  [trailTopology : TopologicalSpace Trail]
  [drawingTopology : TopologicalSpace Drawing]
  (trailMap : Trail  $\approx$  Drawing) : Continuous trailMap
```

SOLUTION

```
exact trailMap.continuous
```

WHY IT WORKS

A homeomorphism carries its continuity proofs with it. The forward proof is the field `trailMap.continuous`.

Ada's drawing now moves continuously with the trail.

Unlocks: tactics `exact`; theorems `Homeomorph.continuous`, `Homeomorph.continuous_symm`, `Homeomorph.symm_apply_apply`, `Homeomorph.trans_apply`

Measured: navigation 49 ms, Lean ready 2 ms, goal preview 6.8 s, verify 74 ms (kernel 19 ms), accepted by the kernel.

2. Her mark lands in the drawing

Ada chooses a point inside the shaded patch and places its mark on the leaf. Since the point is in the part she mapped, the mark must lie in the drawn coordinate region.

Lean names Ada's chosen point `place`. The hypothesis `inPatch : place ∈ chart.source` says that it lies in the shaded part of the stone. Mathlib's `OpenPartialHomeomorph.map_source` then concludes that `chart place` lies in the drawn target. Type `\in` for `∈`.

Two chart facts come along without their own levels: the shaded patch `chart.source` is open, stored as `OpenPartialHomeomorph.open_source`, and the chart is continuous on that patch, stored as `OpenPartialHomeomorph.continuousOn`. Both are in the inventory.

Objective: Show that a point in the chart source maps into its coordinate target.

Think first: Before proving it: why does a chart need a source at all? What goes wrong if Ada tries to draw the whole stone on one leaf?

Answer: A leaf is flat and the stone is closed, so no single continuous one-to-one drawing of the whole stone fits on a leaf. Restricting to a patch is what makes the chart possible.

GOAL

```
theorem local_chart_maps_source {Stone : Type u} {Drawing : Type v}
  [TopologicalSpace Stone] [TopologicalSpace Drawing]
  (chart : OpenPartialHomeomorph Stone Drawing)
  (place : Stone) (inPatch : place ∈ chart.source) :
  chart place ∈ chart.target
```

SOLUTION

```
apply chart.map_source
exact inPatch
```

WHY IT WORKS

Start with `apply chart.map_source`. The remaining goal is the hypothesis `inPatch`.

Once Lean knows that `place` is in the source, its coordinates belong to the target.

Unlocks: tactics `apply`; theorems `OpenPartialHomeomorph.map_source`, `OpenPartialHomeomorph.open_source`, `OpenPartialHomeomorph.continuousOn`

Measured: navigation 41 ms, Lean ready 3 ms, goal preview 798 ms, verify 69 ms (kernel 21 ms), accepted by the kernel.

3. Back to the same spot

Manifold Adventure · Work in progress · One trail, one leaf · Level 3 · Back to the same spot

Games · Worlds · Lean playground · Game source

One trail, one leaf / Level 3 of 4 · LOCAL KERNEL

Back to the same spot

Ada marks a place in her local drawing and traces it back onto the stone. The round trip is reliable only because that place lies inside the patch she drew.

For a partial homeomorphism, the inverse law needs `inPatch : place ∈ chart.source`. This is the formal bridge between "Ada drew this place" and the side condition in Mathlib's `OpenPartialHomeomorph.left_inv`.

`OpenPartialHomeomorph.left_inv`

World tree

- One trail, one leaf 2/4
 - 1. The drawing matches the trail ✓
 - 2. Her mark lands in the drawing ✓
 - 3. Back to the same spot
 - 4. The leaf needs back into the patch
- One path is missing 8/1
- A stack of maps 8/1
- Smooth manifolds 8/1

Objective: Show that a source point returns to itself after passing through the chart and its inverse.

Active goal

Initial goal

Objects

- Stone : Type u
- Drawing : Type v
- instance (TopologicalSpace Stone)
- instance (TopologicalSpace Drawing)
- chart : OpenPartialHomeomorph Stone Drawing
- place : Stone
- inPatch : place ∈ chart.source

Goal: `chart.symm (chart place) = place`

Your proof

Enter tactics only. The surrounding `by` is supplied for you. 0 attempts

by 1

Available tactics

reply | by_cases | calc | constructor

exact | ext | fun_arg | have

infer_instance | intro | left | obtain

rcases | rfl | right | rw | simp

simp | unfold

Verification scope

Ada marks a place in her local drawing and traces it back onto the stone. The round trip is reliable only because that place lies inside the patch she drew.

For a partial homeomorphism, the inverse law needs `inPatch : place ∈ chart.source`. This is the formal bridge between "Ada drew this place" and the side condition in Mathlib's `OpenPartialHomeomorph.left_inv`.

Objective: Show that a source point returns to itself after passing through the chart and its inverse.

GOAL

```
theorem local_chart_round_trip {Stone : Type u} {Drawing : Type v}
  [TopologicalSpace Stone] [TopologicalSpace Drawing]
  (chart : OpenPartialHomeomorph Stone Drawing)
  (place : Stone) (inPatch : place ∈ chart.source) :
  chart.symm (chart place) = place
```

SOLUTION

```
exact chart.left_inv inPatch
```

WHY IT WORKS

The partial round-trip law needs source membership. Give `chart.left_inv` the proof `inPatch`.

Inside her patch, Ada can move from stone to leaf and back without losing her place.

Unlocks: theorems `OpenPartialHomeomorph.left_inv`

Measured: navigation 35 ms, Lean ready 1 ms, goal preview 827 ms, verify 115 ms (kernel 26 ms), accepted by the kernel.

4. The leaf reads back into the patch

Manifold Adventure Work in progress One trail, one leaf · Level 4 The leaf reads back into the patch Games Worlds Lean playground Game source

One trail, one leaf / Level 4 of 4

LOCAL KERNEL

The leaf reads back into the patch

Ada picks a mark inside the drawn region and reads it back onto the stone. Two things should hold at once: the recovered point lies in her shaded patch, and pressing it forward again reproduces the mark she chose.

You have met `chart.map_source` and `chart.left_inv`. Mathlib names their mirror images predictably: `OpenPartialHomeomorph.map_target` and `OpenPartialHomeomorph.right_inv`. Guessing a lemma's name from this convention is a useful Mathlib skill. The goal is a conjunction, written `∧` (type `\and`). The `constructor` tactic splits it into two parts, and a focus dot `·` (type `\.`) gives each part its own proof.

World tree

One trail, one leaf	3/4
1 The drawing matches the trail	✓
2 Her mark lands in the drawing	✓
3 Back to the same spot	✓
4 The leaf reads back into the patch	
One point is missing	8/1
A stack of maps	8/1
Smooth manifolds	8/1

Objective: From `mark ∈ chart.target`, show that the read-back point lies in the source and maps forward to `mark`.

Active goal

Objects

Stone : Type `u`
Drawing : Type `v`
Instance (TopologicalSpace Stone)
Instance (TopologicalSpace Drawing)
chart : OpenPartialHomeomorph Stone Drawing
mark : Drawing
InDrawing : mark ∈ chart.target

Your proof

Enter tactics only. The surrounding `by` is supplied for you. 0 attempts

by 1

Available tactics

reply by_cases calc constructor
 exact ext fun_arg have
 infer_instance intro left obtain
 rcases rfl right rw simp
 simp unfold

► Verification scope

Ada picks a mark inside the drawn region and reads it back onto the stone. Two things should hold at once: the recovered point lies in her shaded patch, and pressing it forward again reproduces the mark she chose.

You have met `chart.map_source` and `chart.left_inv`. Mathlib names their mirror images predictably: `OpenPartialHomeomorph.map_target` and `OpenPartialHomeomorph.right_inv`. Guessing a lemma's name from this convention is a useful Mathlib skill. The goal is a conjunction, written `∧` (type `\and`). The `constructor` tactic splits it into two parts, and a focus dot `·` (type `\.`) gives each part its own proof.

Objective: From `mark ∈ chart.target`, show that the read-back point lies in the source and maps forward to `mark`.

GOAL

```
theorem local_chart_reads_back {Stone : Type u} {Drawing : Type v}
  [TopologicalSpace Stone] [TopologicalSpace Drawing]
  (chart : OpenPartialHomeomorph Stone Drawing)
  (mark : Drawing) (inDrawing : mark ∈ chart.target) :
  chart.symm mark ∈ chart.source ∧ chart (chart.symm mark) = mark
```

SOLUTION

```
constructor
· exact chart.map_target inDrawing
· exact chart.right_inv inDrawing
```

WHY IT WORKS

Split the conjunction, then handle each goal after a focus dot. Use `constructor`; the mirror lemmas are `chart.map_target` and `chart.right_inv`.

Both directions of the chart now behave, and Ada guessed the lemma names herself.

Unlocks: tactics `constructor`, `·`; theorems `OpenPartialHomeomorph.map_target`, `OpenPartialHomeomorph.right_inv`

Measured: navigation 67 ms, Lean ready 1 ms, goal preview 795 ms, verify 111 ms (kernel 28 ms), accepted by the kernel.

One pole is missing

Manifold Adventure

WIP | Work in progress

Games

Worlds

Lean playground

Game source

Back to world tree

LOCAL KERNEL

One pole is missing

0/5 complete

A round world on a flat leaf

Ada finds a glass bead near the trail. She wants to copy its surface onto a leaf, but one drawing cannot include the point where she holds the bead. She makes a second drawing from the other pole to cover the gap. This is the whole reason manifolds need atlases, so the course meets it before it meets the general words.

Mathlib builds the drawing as `stereographic`, an `OpenPartialHomeomorph` from the unit sphere to a flat plane. Try the projection lab below: drag Ada around the bead and watch her mark race off the leaf as she nears the pole, then switch poles and compare the two readings of the same point. The levels prove what the lab shows, and introduce the case-splitting tactics they need on the way to the covering proof.

Projection lab

Stereographic projection of a circle

North leaf

South leaf

Both

Ada's position on the bead48°

Walk near the north pole

Stand on the south pole

North pole

01

The pole stays off the leaf

Local kernel

02

The far pole lands in the middle

Local kernel

03

Off the pole, onto the leaf

Local kernel

04

One of the two leaves shows her place

Local kernel

05

The second leaf covers the hole

Local kernel

A round world on a flat leaf

Ada finds a glass bead near the trail. She wants to copy its surface onto a leaf, but one drawing cannot include the point where she holds the bead. She makes a second drawing from the other pole to cover the gap. This is the whole reason manifolds need atlases, so the course meets it before it meets the general words.

Mathlib builds the drawing as `stereographic`, an `OpenPartialHomeomorph` from the unit sphere to a flat plane. Try the projection lab below: drag Ada around the bead and watch her mark race off the leaf as she nears the pole, then switch poles and compare the two readings of the same point. The levels prove what the lab shows, and introduce the case-splitting tactics they need on the way to the covering proof.

Measured: world page opened in 26 ms.

The Manifold Adventure r5 · page 14 of 75

5. The pole stays off the leaf

Ada presses one point of the bead between her feet while she draws. That point is the pole of the projection, so it cannot appear in this chart.

The source of `stereographic unitPole` is the sphere with the pole removed. Mathlib states this as `stereographic_source`. The complement notation $\{(pole, \dots)\}^c$ means every sphere point except that one.

The singleton's element appears as $\{pole, by simp [unitPole]\}$. A sphere point is a vector paired with a certificate that its norm is one. Read the embedded proof as Mathlib filling in that certificate from `unitPole`.

One more fact rides along: every mark on the leaf comes from some point of the bead, `surjective_stereographic`, so no coordinate on the drawing is wasted. It is in the inventory.

Objective: Show that the stereographic chart covers the sphere except for its chosen pole.

Think first: Before proving it: as Ada walks toward the pole she is projecting from, what happens to her mark on the leaf?

Answer: It runs off toward infinity. The pole itself has no mark, which is why it must be removed from the chart's source.

GOAL

```
theorem stereographic_map_misses_pole {Space : Type u} [NormedAddCommGroup Space]
  [InnerProductSpace ℝ Space]
  (pole : Space) (unitPole : ||pole|| = 1) :
  (stereographic unitPole).source = {(pole, by simp [unitPole])}^c
```

SOLUTION

```
exact stereographic_source unitPole
```

WHY IT WORKS

Mathlib states the source of a stereographic chart directly. Use `stereographic_source unitPole`.

The first drawing now has an exact missing point.

Unlocks: theorems `stereographic_source`, `surjective_stereographic`

Measured: navigation 39 ms, Lean ready 3 ms, goal preview 1.8 s, verify 112 ms (kernel 52 ms), accepted by the kernel.

6. The far pole lands in the middle

One pole is missing / Level 2 of 5

The far pole lands in the middle

Ada marks the point opposite the missing pole. On her flat drawing, that point sits at the center.

The value `!pole` is the antipodal point on the sphere. Mathlib's `stereographic_apply_neg` computes its stereographic coordinate as the zero vector in the flat model space.

The pole has changed representation since the previous level. There it was a raw vector plus `||pole|| = 1`. Here it is already a point of the sphere subtype, and `norm_eq_of_mem_sphere pole` recovers the norm certificate from membership.

World tree

- One ball, one leaf 4/4
- One pole is missing 1/3
 - The pole stays off the leaf ✓
 - The far pole lands in the middle**
 - Off the pole, onto the leaf
 - One of the two leaves shows her place
 - The second leaf covers the hole

A stack of maps 0/1

Objective: Prove that the antipodal point maps to the origin of the drawing.

Active goal

Initial goal

Objects

Space : Type u

Instance (NormedAddCommGroup Space)

Instance (InnerProductSpace ℝ Space)

pole : sphere (0 : Space) 1

stereographic

→ (norm_eq_of_mem_sphere pole) (-pole) = 0

Your proof

Error: tactic only. The surrounding key is supplied for you.

8 attempts

by 1

Verify answer Show a hint View solution

Available tactics

apply by_cases calc constructor exact ext fun_arg have infer_instance intro left obtain rcases rfl right rw simp simp unfold

Verification scope

Ada marks the point opposite the missing pole. On her flat drawing, that point sits at the center.

The value `-pole` is the antipodal point on the sphere. Mathlib's `stereographic_apply_neg` computes its stereographic coordinate as the zero vector in the flat model space.

The pole has changed representation since the previous level. There it was a raw vector plus `||pole|| = 1`. Here it is already a point of the sphere subtype, and `norm_eq_of_mem_sphere pole` recovers the norm certificate from membership.

Objective: Prove that the antipodal point maps to the origin of the drawing.

GOAL

```
theorem opposite_pole_is_origin {Space : Type u} [NormedAddCommGroup Space]
  [InnerProductSpace ℝ Space]
  (pole : sphere (0 : Space) 1) :
    stereographic (norm_eq_of_mem_sphere pole) (-pole) = 0
```

SOLUTION

```
exact stereographic_apply_neg pole
```

WHY IT WORKS

The antipode's coordinate is a named computation. Apply `stereographic_apply_neg` to `pole`.

Ada can use the opposite pole as the center of her coordinates.

Unlocks: theorems `stereographic_apply_neg`, `norm_eq_of_mem_sphere`

Measured: navigation 34 ms, Lean ready 2 ms, goal preview 794 ms, verify 214 ms (kernel 85 ms), accepted by the kernel.

7. Off the pole, onto the leaf

The screenshot shows the Lean 4 IDE interface for the 'Off the pole, onto the leaf' level. The left sidebar displays the level's progress, including a list of tasks and a stack of maps. The main workspace shows the current goal: 'Show that any point other than the pole lies in that pole's chart source.' The objects section lists variables like `Space`, `Instance`, and `Assumptions`. The right sidebar shows available tactics such as `apply`, `by_cases`, `calc`, `constructor`, `exact`, `ext`, `fun_undef`, `have`, `infer_instance`, `intro`, `left`, `obtain`, `rcases`, `rw`, `right`, `rw`, `simp`, and `unfold`.

Ada checks a point that is not the pinned pole. Every such point earned a mark on the first leaf.

Membership in the source is membership in a complement. The condition $\text{place} \in \{\text{north}\}^c$ becomes $\text{place} \notin \{\text{north}\}$, and singleton membership becomes equality, so the whole condition is $\text{place} \neq \text{north}$. The tactic `simp only [h1, h2, ...]` rewrites with exactly the listed lemmas. Here those lemmas include `stereographic_source`, `Set.mem_compl_iff`, and `Set.mem_singleton_iff`.

Objective: Show that any point other than the pole lies in that pole's chart source.

GOAL

```
theorem stereographic_off_pole {Space : Type u} [NormedAddCommGroup Space]
  [InnerProductSpace ℝ Space]
  (north place : sphere (0 : Space) 1) (notNorth : place ≠ north) :
  place ∈ (stereographic (norm_eq_of_mem_sphere north)).source
```

SOLUTION

```
simp only [stereographic_source, Set.mem_compl_iff, Set.mem_singleton_iff]
exact notNorth
```

WHY IT WORKS

Unwind the chart source and the two membership statements. Use `simp only [stereographic_source, Set.mem_compl_iff, Set.mem_singleton_iff]`; the result is `notNorth`.

Only the pinned pole is missing. Everything else already has coordinates on the first leaf.

Unlocks: tactics `simp`; theorems `Set.mem_compl_iff`, `Set.mem_singleton_iff`

Measured: navigation 34 ms, Lean ready 1 ms, goal preview 791 ms, verify 114 ms (kernel 58 ms), accepted by the kernel.

8. One of the two leaves shows her place

The screenshot shows the Manifold Adventure interface. On the left, a sidebar contains a 'World tree' with a list of goals: 'One leaf, one leaf' (4/4), 'One pole is missing' (3/3), and 'One of the two leaves shows her place' (4/4). The main area displays the 'Objective: Show that every point of the bead lies in the source of the north chart or in the source of the south chart.' Below this, the 'Active goal' is shown with variables u , v , and $place$. The goal is to prove that $place \in \text{stereographic}(\text{norm_eq_of_mem_sphere } north).source \vee place \in \text{stereographic}(\text{norm_eq_of_mem_sphere } south).source$. The 'Assumptions' section lists $different : north \neq south$. The 'Your proof' section shows a tactic `by 1` applied to the goal.

Ada holds two drawings of the bead, one made from the north pole and one from the south pole. She picks any point of the bead and asks which drawing shows it. If the point is the north pole, the south drawing has it. Otherwise the north drawing does.

Each source is the sphere with one pole removed, so after `stereographic_source` and the membership lemmas from the previous level, the goal reads "`place  $\neq$  north` or `place  $\neq$  south`". A proof by cases on `place = north` (the tactic `by_cases`) splits the world in two. In each half, `left` or `right` chooses which side of the `v` to prove. To prove `place  $\neq$  south`, which means `place = south  $\rightarrow$  False`, use `intro` to assume the equality and derive the contradiction with `different`, chaining the two equalities as `atNorth.symm.trans atSouth`.

Objective: Show that every point of the bead lies in the source of the north chart or in the source of the south chart.

Think first: Before proving it: which single point of the bead is missing from the north drawing, and why can the south drawing not miss that same point?

Answer: The north drawing misses exactly the north pole. The south drawing misses exactly the south pole, and the two poles are different points, so no point is missing from both.

GOAL

```
theorem two_leaves_cover_point {Space : Type u} [NormedAddCommGroup Space]
  [InnerProductSpace ℝ Space]
  (north south : sphere (0 : Space) 1) (different : north  $\neq$  south)
  (place : sphere (0 : Space) 1) :
  place  $\in$  (stereographic (norm_eq_of_mem_sphere north)).source  $\vee$ 
  place  $\in$  (stereographic (norm_eq_of_mem_sphere south)).source
```

SOLUTION

```
simp only [stereographic_source, Set.mem_compl_iff, Set.mem_singleton_iff]
by_cases atNorth : place = north
· right
  intro atSouth
  exact different (atNorth.symm.trans atSouth)
· left
  exact atNorth
```

WHY IT WORKS

Split on whether the point is the north pole. In each case one of the two leaves must show it. After `simp only` with the source and membership lemmas, use `by_cases atNorth : place = north`, then `right` or `left`; the north case needs `intro` and `different (atNorth.symm.trans atSouth)`.

Every point of the bead appears on at least one of Ada's two leaves.

Unlocks: tactics `by_cases`, `left`, `right`, `intro`; theorems `Eq.symm`, `Eq.trans`

Measured: navigation 39 ms, Lean ready 1 ms, goal preview 794 ms, verify 227 ms (kernel 79 ms), accepted by the kernel.

9. The second leaf covers the hole

The screenshot shows the Lean 4 IDE interface for the 'The second leaf covers the hole' level. The main area displays the objective: 'Prove that two stereographic charts with different poles cover the whole sphere.' The 'Your proof' section shows a partial proof starting with 'by 1'. The right sidebar lists available tactics such as 'apply', 'by_cases', 'calc', 'constructor', 'exact', 'ext', 'fun_ext', 'have', 'infer_instance', 'intro', 'left', 'obtain', 'rcases', 'rfl', 'right', 'rw', 'simp', and 'unfold'.

Ada makes a second drawing from a different pole. The first leaf misses only the north point, and the second misses only the south point. Since those points differ, every place appears on at least one leaf.

The proof starts from `stereographic_source` and has a useful shape. `ext place` turns the set equation into a statement about one point. `simp only` with the membership lemmas from the previous level reduces it to a disjunction: the point differs from north, or it differs from south. `by_cases atNorth : place = north` splits the situations, while `left` and `right` choose a side. At north, assume `place = south` with `intro` and chain equalities as `atNorth.symm.trans atSouth` to contradict `different`.

Objective: Prove that two stereographic charts with different poles cover the whole sphere.

GOAL

```
theorem two_stereographic_maps_cover {Space : Type u} [NormedAddCommGroup Space]
  [InnerProductSpace ℝ Space]
  (north south : sphere (0 : Space) 1) (different : north ≠ south) :
  (stereographic (norm_eq_of_mem_sphere north)).source u
    (stereographic (norm_eq_of_mem_sphere south)).source = Set.univ
```

SOLUTION

```
ext place
simp only [stereographic_source, Set.mem_union, Set.mem_compl_iff,
  Set.mem_singleton_iff, Set.mem_univ, iff_true]
by_cases atNorth : place = north
· right
  intro atSouth
  exact different (atNorth.symm.trans atSouth)
· left
  exact atNorth
```

WHY IT WORKS

Reduce the set equality to one point, expose the disjunction, then split on whether that point is north. Use `ext`, `simp only`, and `by_cases`. Choose branches with `left` or `right`.

Two leaves are enough to record every point on the bead. This pair is the atlas Mathlib uses to make the sphere a `ChartedSpace`, which the next world states in general.

Unlocks: tactics `ext`; theorems `Set.mem_union`, `Set.mem_univ`, `iff_true`

Measured: navigation 33 ms, Lean ready 2 ms, goal preview 797 ms, verify 128 ms (kernel 70 ms), accepted by the kernel.

A stack of maps

Manifold Adventure

WIP | Work in progress

Games

Worlds

Lean playground

Game source

Back to world tree

LOCAL KERNEL

A stack of maps

0/5 complete

A stack of maps

The bead needed two leaves; a rougher stone may need many. Ada keeps her leaves together as her atlas, and for every place she stands she has a preferred leaf that shows it.

The class `ChartedSpace` equips a surface with an atlas and a preferred chart `chartAt` for each point. The goals call the actual world `Surface`, the shared coordinate space `Coordinates`, and Ada's location `place`. Mathlib often writes the same three objects as `M`, `H`, and `x`. The last two levels return to the bead and identify its preferred leaves with the projections of the previous world. The angle lab below shows the same idea on a circle: two arcs, each with its own angle reading, and an overlap where the readings differ by a fixed shift.

Two arcs, two angle charts

Angles on a dial

Angle fed to Circle.exp0.83π

Add a full turnRemove a full turn

01A leaf for where she standsLocal kernel

02Her place lands on the leafLocal kernel

03No place left uncoveredLocal kernel

04The preferred leaf comes from the far poleLocal kernel

05Every projection is filed in the atlasLocal kernel

A stack of maps

The bead needed two leaves; a rougher stone may need many. Ada keeps her leaves together as her atlas, and for every place she stands she has a preferred leaf that shows it.

The class `ChartedSpace` equips a surface with an atlas and a preferred chart `chartAt` for each point. The goals call the actual world `Surface`, the shared coordinate space `Coordinates`, and Ada's location `place`. Mathlib often writes the same three objects as `M`, `H`, and `x`. The last two levels return to the bead and identify its preferred leaves with the projections of the previous world. The angle lab below shows the same idea on a circle: two arcs, each with its own angle reading, and an overlap where the readings differ by a fixed shift.

Measured: world page opened in 22 ms.

The Manifold Adventure r5 · page 22 of 75

10. A leaf for where she stands

The screenshot shows the Lean 4 IDE interface. On the left, there's a sidebar with a list of levels and a description of the problem. The main area displays the problem statement and the goal. The goal is to show that a specific point belongs to the source of a chart chosen from a collection.

Problem Statement: Wherever Ada stops, she selects a leaf whose shaded patch contains her current position. A preferred map that missed her would be useless.

Goal: Show that `place` lies in the source of the chart chosen there.

Available tactics: `apply`, `by_cases`, `calc`, `constructor`, `exact`, `ext`, `fun_lemma`, `have`, `infer_instance`, `intro`, `left`, `obtain`, `rcases`, `refl`, `right`, `rw`, `simp`, `simp`, `unfold`.

Goal: `place ∈ (chartAt Coordinates place).source`

Proof: `by 1`

Wherever Ada stops, she selects a leaf whose shaded patch contains her current position. A preferred map that missed her would be useless.

The instance `[ChartedSpace Coordinates Surface]` is Ada's collection of local leaves. Mathlib's `mem_chart_source` says that `chartAt Coordinates place`, the leaf chosen at her current location, contains `place` in its source. Smooth compatibility between overlapping leaves comes later, with `IsManifold`.

Two neighbouring facts join the inventory here: the preferred leaf is one of the leaves in the atlas, `chart_mem_atlas`, and its shaded patch contains a whole neighbourhood of `place`, `chart_source_mem_nhds`. The second is stated with the neighbourhood filter `ℳ place`, which is a collection of sets, so `source ∈ ℳ place` says that the source contains an open set around `place`.

Objective: Show that `place` lies in the source of the chart chosen there.

GOAL

```
theorem point_mem_preferred_chart {Coordinates : Type u} {Surface : Type v}
  [TopologicalSpace Coordinates] [TopologicalSpace Surface]
  [ChartedSpace Coordinates Surface] (place : Surface) :
  place ∈ (chartAt Coordinates place).source
```

SOLUTION

```
exact mem_chart_source Coordinates place
```

WHY IT WORKS

The preferred chart is built not to miss its chosen point. Use `mem_chart_source Coordinates place`.

Ada can always choose a chart that contains where she stands.

Unlocks: theorems `mem_chart_source`, `chart_mem_atlas`, `chart_source_mem_nhds`

Measured: navigation 38 ms, Lean ready 8 ms, goal preview 792 ms, verify 63 ms (kernel 17 ms), accepted by the kernel.

11. Her place lands on the leaf

Manifold Adventure · Work in progress · A stack of maps · Level 2 · Her place lands on the leaf

Her place lands on the leaf

Ada presses her current position through the chosen chart. Its mark lands inside the coordinate patch drawn on the leaf.

Read `chartAt Coordinates place place` as `(chartAt Coordinates place) place`. The first `place` selects Ada's chart, and the second is the point drawn in `Coordinates`. No new lemma is needed. The first world's `map_source` sends a source point into its chart's target, and the first level of this world put `place` in this chart's source. Mathlib also packages the combination as `mem_chart_target`, which joins your book after this proof.

Objective: Show that the coordinates of `place` lie inside the chosen chart's target.

Active goal

Objects

- `Coordinates` : Type `u`
- `Surface` : Type `v`
- `Instance` (TopologicalSpace Coordinates)
- `Instance` (TopologicalSpace Surface)
- `Instance` (ChartedSpace Coordinates Surface)
- `place` : Surface

chartAt Coordinates place place

→ `0 (chartAt Coordinates place).target`

Your proof

Enter tactics only. The surrounding `by` is supplied for you.

by 1

Available tactics

apply | by_cases | calc | constructor

exact | ext | fun_arg | have

infer_instance | intro | left | obtain

rcases | rfl | right | rw | simp

simp | unfold

► Verification scope

Level rewards

Now this time

World time

One trail, one leaf 4/4

One pole is missing 5/5

A stack of maps 1/5

1 A leaf for where she starts ✓

2 **Her place lands on the leaf**

3 No place left uncovered

4 The preferred leaf comes from the far pole

5 Every projection is fixed in the atlas

Ada presses her current position through the chosen chart. Its mark lands inside the coordinate patch drawn on the leaf.

Read `chartAt Coordinates place place` as `(chartAt Coordinates place) place`. The first `place` selects Ada's chart, and the second is the point drawn in `Coordinates`. No new lemma is needed. The first world's `map_source` sends a source point into its chart's target, and the first level of this world put `place` in this chart's source. Mathlib also packages the combination as `mem_chart_target`, which joins your book after this proof.

Objective: Show that the coordinates of `place` lie inside the chosen chart's target.

GOAL

```
theorem preferred_chart_maps_to_target {Coordinates : Type u} {Surface : Type v}
  [TopologicalSpace Coordinates] [TopologicalSpace Surface]
  [ChartedSpace Coordinates Surface] (place : Surface) :
  chartAt Coordinates place place ∈ (chartAt Coordinates place).target
```

SOLUTION

```
apply (chartAt Coordinates place).map_source
exact mem_chart_source Coordinates place
```

WHY IT WORKS

Combine the chart law `map_source` with the fact that the preferred chart contains its point. Apply `(chartAt Coordinates place).map_source`; the remaining goal is `mem_chart_source Coordinates place`.

Ada built the target fact from parts she already owned. Mathlib's one-step `mem_chart_target` is now in her book too.

Unlocks: theorems `mem_chart_target`

Measured: navigation 36 ms, Lean ready 2 ms, goal preview 790 ms, verify 62 ms (kernel 20 ms), accepted by the kernel.

12. No place left uncovered

Objective: Show that the preferred chart sources cover every point of `Surface`.

Active goal:

```

theorem preferred_charts_cover {Coordinates : Type u} {Surface : Type v}
  [TopologicalSpace Coordinates] [TopologicalSpace Surface]
  [ChartedSpace Coordinates Surface] :
  (⋃ place : Surface, (chartAt Coordinates place).source) =
    (Set.univ : Set Surface)

```

Objects:

- `Coordinates : Type u`
- `Surface : Type v`
- `Instance (TopologicalSpace Coordinates)`
- `Instance (TopologicalSpace Surface)`
- `Instance (ChartedSpace Coordinates Surface)`

Available tactics:

- `apply`, `by_cases`, `calc`, `constructor`, `exact`, `ext`, `fun_undef`, `have`, `infer_instance`, `intro`, `left`, `obtain`, `rcases`, `refl`, `right`, `rw`, `simp`, `simplify`, `unfold`

Your proof:

```

by
  1

```

Ada spreads every preferred leaf across the stone. No point remains uncovered; wherever she stands, at least one local map is ready.

The indexed union $\bigcup \text{place} : \text{Surface}, (\text{chartAt } \text{Coordinates } \text{place}).\text{source}$ spreads out the preferred leaf at every possible location. Mathlib proves in `iUnion_source_chartAt` that their sources equal all of `Surface`.

Objective: Show that the preferred chart sources cover every point of `Surface`.

GOAL

```

theorem preferred_charts_cover {Coordinates : Type u} {Surface : Type v}
  [TopologicalSpace Coordinates] [TopologicalSpace Surface]
  [ChartedSpace Coordinates Surface] :
  (⋃ place : Surface, (chartAt Coordinates place).source) =
    (Set.univ : Set Surface)

```

SOLUTION

```

exact iUnion_source_chartAt Coordinates Surface

```

WHY IT WORKS

Every point lies in its own preferred chart, so their union is universal. Use `iUnion_source_chartAt Coordinates Surface`.

Together, Ada's leaves cover the whole space.

Unlocks: theorems `iUnion_source_chartAt`

Measured: navigation 70 ms, Lean ready 2 ms, goal preview 1.8 s, verify 95 ms (kernel 19 ms), accepted by the kernel.

13. The preferred leaf comes from the far pole

The screenshot shows the Manifold Adventure interface. On the left, a sidebar contains a progress list with 5 items, where the 4th item, 'The preferred leaf comes from the far pole', is selected. The main area displays a proof goal: 'Objective: Show that the chart Mathlib prefers at $-place$ is the stereographic projection from the opposite point.' Below the goal, the 'Active goal' section shows the current state of the proof, including objects like `Space`, `Instance`, `dimension`, and `place`, and assumptions like `Fact (Module.finiteRank R Space = dimension + 1)`. The 'Your proof' section shows a single tactic `by 1`. On the right, a panel lists 'Available tactics' such as `apply`, `by_cases`, `calc`, `constructor`, `exact`, `ext`, `fun_ext`, `have`, `infer_instance`, `intro`, `left`, `obtain`, `rcases`, `rfl`, `right`, `rw`, `simp`, `simp`, and `unfold`.

Ada stands on the bead and asks for the leaf her atlas prefers at her location. Mathlib hands her the projection made from the point directly opposite her, so that her own place lands near the middle of the drawing.

The sphere's `ChartedSpace` instance in Mathlib chooses `chartAt` to be `stereographic'` from the antipodal point `-place`. Here `stereographic'` is the same projection as before, rewritten to land in `EuclideanSpace ℝ (Fin dimension)`, the ordinary coordinate space of the right dimension. The assumption `Fact (Module.finiteRank R Space = dimension + 1)` records that a sphere inside a space of dimension `dimension + 1` is itself `dimension`-dimensional. Because this is how the instance is defined, the two sides are equal by definition, and `rfl` closes the goal.

Objective: Show that the chart Mathlib prefers at `place` is the stereographic projection from the opposite point.

Think first: Before proving it: if Ada stands at the north pole, which point is the pole of her preferred chart, and where does she herself land on that drawing?

Answer: Her preferred chart projects from the south pole. She stands opposite that pole, so her own position lands at the origin of the drawing.

GOAL

```
theorem sphere_preferred_chart {Space : Type u} [NormedAddCommGroup Space]
  [InnerProductSpace ℝ Space] {dimension : ℕ}
  [Fact (Module.finiteRank ℝ Space = dimension + 1)]
  (place : sphere (0 : Space) 1) :
  chartAt (EuclideanSpace ℝ (Fin dimension)) place = stereographic' dimension (-place)
```

SOLUTION

```
rfl
```

WHY IT WORKS

The instance was defined by this very formula. Definitional equalities close with `rfl`.

The abstract `chartAt` and the concrete projection are the same leaf.

Unlocks: tactics `rfl`

Measured: navigation 46 ms, Lean ready 2 ms, goal preview 791 ms, verify 210 ms (kernel 79 ms), accepted by the kernel.

14. Every projection is filed in the atlas

The screenshot shows the Manifold Adventure interface. The main goal is "Every projection is filed in the atlas". The interface includes a sidebar with a progress bar, a main area with the goal and available tactics, and a bottom area with a list of tasks.

Objective: Show that the projection from any pole belongs to the sphere's atlas.

Available tactics: apply, by_cases, calc, constructor, exact, ext, fun_arg, have, infer_instance, intro, left, obtain, rcases, rfl, right, rw, simp, simp_unfold, ...

Level rewards:

- One trail, one leaf: 4/4
- One pole is missing: 5/5
- A stack of maps: 4/5
- 1. A leaf for where she starts: ✓
- 2. Her place lands on the leaf: ✓
- 3. No place left uncovered: ✓
- 4. The preferred leaf comes from the far pole: ✓
- 5. Every projection is filed in the atlas: ✓

Ada can project the bead from any pole she likes. Each such drawing belongs in her atlas, not only the two she happened to make first.

Mathlib defines the sphere's atlas as the set of all charts of the form `stereographic' dimension pole` for some pole: in Lean, membership in `atlas` unfolds to the statement "there exists a pole with `stereographic' dimension pole = stereographic' dimension pole`". An existence statement is proved by supplying a witness and a proof, written with angle brackets as `{pole, rfl}`.

Objective: Show that the projection from any pole belongs to the sphere's atlas.

GOAL

```
theorem sphere_chart_in_atlas {Space : Type u} [NormedAddCommGroup Space]
  [InnerProductSpace ℝ Space] {dimension : ℕ}
  [Fact (Module.finite ℝ Space = dimension + 1)]
  (pole : sphere (0 : Space) 1) :
    stereographic' dimension pole ∈ atlas (EuclideanSpace ℝ (Fin dimension)) (sphere (0 :
    Space) 1)
```

SOLUTION

```
exact {pole, rfl}
```

WHY IT WORKS

The atlas is defined as "all projections from some pole", so exhibit the pole. Use the anonymous constructor `{pole, rfl}` with `exact`.

Ada's atlas holds one leaf for every pole she could choose.

Measured: navigation 31 ms, Lean ready 1 ms, goal preview 791 ms, verify 107 ms (kernel 56 ms), accepted by the kernel.

Smooth manifolds

Manifold Adventure

WIP Work in progress

Games

Worlds

Lean playground

Game source

Back to world tree

LOCAL KERNEL

Smooth manifolds

0/5 complete

When chart changes are smooth

Ada's leaves overlap, so she can compare two coordinate drawings of the same place. Continuity keeps nearby points nearby, but calculus also needs the change between drawings to have controlled derivatives. That change is the only place where "smooth" can be defined, because the surface itself has no coordinates.

Mathlib's `IsManifold` adds exactly this condition to a `ChartedSpace`. Three words appear in the goals and deserve a plain reading. `Scalar` is the number field; read it as $\mathbb{R}$. `ModelWithCorners` connects the coordinate space to the vector space where derivatives live; for ordinary Euclidean coordinates it is the identity, written $\mathcal{R}^n$ or $\mathcal{I}(\mathbb{R}, E)$, and corners only matter for manifolds with boundary. `order : WithTop  $\mathbb{N}_\infty$` is how many derivatives are required: a number, ∞ for all finite orders, or ω for analytic. The world states the definition in general, checks it on the bead, and ends with the torus as a product.

01

Two leaves in conversation

Local kernel

02

Changing leaves is a smooth move

Local kernel

03

The smooth move, spelled out in calculus

Local kernel

04

Two drawings of the bead agree smoothly

Local kernel

05

Two circles make a torus

Local kernel

Convenience port of Lean4Game; upstream creators and licenses retain credit.

Mathlib proof checking stays in this browser.

When chart changes are smooth

Ada's leaves overlap, so she can compare two coordinate drawings of the same place. Continuity keeps nearby points nearby, but calculus also needs the change between drawings to have controlled derivatives. That change is the only place where "smooth" can be defined, because the surface itself has no coordinates.

Mathlib's `IsManifold` adds exactly this condition to a `ChartedSpace`. Three words appear in the goals and deserve a plain reading. `Scalar` is the number field; read it as $\mathbb{R}$. `ModelWithCorners` connects the coordinate space to the vector space where derivatives live; for ordinary Euclidean coordinates it is the identity, written $\mathcal{R}^n$ or $\mathcal{I}(\mathbb{R}, E)$, and corners only matter for manifolds with boundary. `order : WithTop  $\mathbb{N}_\infty$` is how many derivatives are required: a number, ∞ for all finite orders, or ω for analytic. The world states the definition in general, checks it on the bead, and ends with the torus as a product.

Measured: world page opened in 29 ms.

15. Two leaves in conversation

Smooth manifolds / Level 1 of 5

Two leaves in conversation

Ada holds two overlapping leaves of the same stone. She reads a mark from the first leaf back onto the stone, then presses that point through the second. This is the transition map between the drawings.

Formally, the transition map is `chart.symm.trans chart' : invert one chart, then apply the other. Its domain contains marks in the first chart's target whose read-back lands in the second chart's source. The preimage chart.symm-1 chart'.source collects those marks. Mathlib computes the domain with OpenPartialHomeomorph.trans_source, while OpenPartialHomeomorph.symm_source renames the inverse chart's source to chart.target.`

World tree

One leaf, one leaf	4/4
One point is missing	5/5
A stack of maps	5/5
Smooth manifolds	6/5

1 Two leaves in conversation

2 Changing leaves is a smooth move

3 The smooth move, spelled out in calculus

4 Two drawings of the leaf agree smoothly

Objective: Compute the transition map's domain from the two charts.

Active goal

```

(chart.symm.trans chart').source
=
chart.target n chart.symm-1 chart'.source

```

Objects

```

Stone : Type u
Drawing : Type v
instance [TopologicalSpace Stone] [TopologicalSpace Drawing]
chart chart' : OpenPartialHomeomorph Stone Drawing

```

Your proof

```

by
  1.

```

Available tactics

apply | by_cases | calc | constructor

exact | ext | fun_arg | have

infer_instance | intro | left | obtain

rcases | rfl | right | rw | simp

simp | unfold |

► Verification scope

Ada holds two overlapping leaves of the same stone. She reads a mark from the first leaf back onto the stone, then presses that point through the second. This is the transition map between the drawings.

Formally, the transition map is `chart.symm.trans chart' : invert one chart, then apply the other. Its domain contains marks in the first chart's target whose read-back lands in the second chart's source. The preimage chart.symm-1 chart'.source collects those marks. Mathlib computes the domain with OpenPartialHomeomorph.trans_source, while OpenPartialHomeomorph.symm_source renames the inverse chart's source to chart.target.`

Two related facts join the inventory: a smooth atlas is also a continuous atlas, so `IsManifold model 0 Surface` follows from any higher order, and the model coordinate space is itself a manifold at every order, `instIsManifoldModelSpace`. The optional "How smooth is smooth" world proves these.

Objective: Compute the transition map's domain from the two charts.

GOAL

```

theorem transition_map_source {Stone : Type u} {Drawing : Type v}
  [TopologicalSpace Stone] [TopologicalSpace Drawing]
  (chart chart' : OpenPartialHomeomorph Stone Drawing) :
  (chart.symm.trans chart').source =
    chart.target n chart.symm-1 chart'.source

```

SOLUTION

```

rw [OpenPartialHomeomorph.trans_source, OpenPartialHomeomorph.symm_source]

```

WHY IT WORKS

Unfold the source of the composite, then rename the inverse chart's source. Rewrite with `OpenPartialHomeomorph.trans_source` and `OpenPartialHomeomorph.symm_source`.

The transition map now has an explicit home: the overlap as seen from the first leaf.

Unlocks: tactics `rw`; theorems `OpenPartialHomeomorph.trans_source`, `OpenPartialHomeomorph.symm_source`, `instIsManifoldModelSpace`

Measured: navigation 30 ms, Lean ready 2 ms, goal preview 792 ms, verify 117 ms (kernel 27 ms), accepted by the kernel.

16. Changing leaves is a smooth move

Objective: Show that the change of coordinates between two atlas charts belongs to the smooth groupoid.

Available tactics: apply, by_cases, calc, constructor, exact, ext, fun_ext, have, infer_instance, intro, left, obtain, rcases, rfl, right, rw, simp, simp, unfold, ...

World tree:

World tree	Progress
One trail, one leaf	4/4
One pole is missing	3/3
A stack of maps	5/5
Smooth manifolds	1/5
1. Two leaves in conversation	✓
2. Changing leaves is a smooth move	
3. The smooth move, spelled out in calculus	
4. Two drawings of the best agree smoothly	

Your proof: Enter tactics only. The surrounding `by` is supplied for you. 0 attempts

Ada compares two leaves from her atlas over the region where both apply. Reading a mark from the first leaf back onto the surface and then onto the second leaf is the transition between the two drawings. On a smooth manifold, this transition never creases.

This level is the definition of a smooth manifold. `IsManifold` says the atlas has a structure groupoid: every change of charts `chart.symm >>_h chart'` between atlas members belongs to `contDiffGroupoid`, the collection of local maps of the coordinate space that are differentiable to the given `order`. The symbol `>>_h` composes partial homeomorphisms. The field `HasGroupoid.compatible` states exactly this, given the two atlas memberships.

Objective: Show that the change of coordinates between two atlas charts belongs to the smooth groupoid.

Think first: Before proving it: the surface itself has no coordinates. Where does the derivative that "smooth" talks about actually live?

Answer: In the coordinate space. Smoothness is a property of the transition maps between charts, which are maps between open sets of the coordinate space, where derivatives make sense.

GOAL

```
theorem chart_change_is_smooth {Scalar : Type u} [NontriviallyNormedField Scalar]
  {Vectors : Type v} [NormedAddCommGroup Vectors]
  [NormedSpace Scalar Vectors]
  {Coordinates : Type w} [TopologicalSpace Coordinates]
  {model : ModelWithCorners Scalar Vectors Coordinates}
  {Surface : Type u} [TopologicalSpace Surface]
  [ChartedSpace Coordinates Surface]
  {order : WithTop ℕ} [IsManifold model order Surface]
  (chart chart' : OpenPartialHomeomorph Surface Coordinates)
  (inAtlas : chart ∈ atlas Coordinates Surface)
  (inAtlas' : chart' ∈ atlas Coordinates Surface) :
  chart.symm >>_h chart' ∈ contDiffGroupoid order model
```

SOLUTION

```
exact HasGroupoid.compatible inAtlas inAtlas'
```

WHY IT WORKS

A smooth manifold is, by definition, an atlas whose chart changes are smooth. The instance field `HasGroupoid.compatible` takes the two atlas memberships.

The transition between any two of Ada's leaves is as smooth as the manifold promises.

Unlocks: theorems `HasGroupoid.compatible`

Measured: navigation 39 ms, Lean ready 2 ms, goal preview 1.3 s, verify 112 ms (kernel 33 ms), accepted by the kernel.

17. The smooth move, spelled out in calculus

The screenshot shows the Lean Playground interface for the exercise "The smooth move, spelled out in calculus". The interface is divided into several sections:

- Left sidebar:** Contains a "Smooth manifolds" section with a progress bar (2/5) and a list of exercises. The current exercise is "The smooth move, spelled out in calculus".
- Top bar:** Shows the current level (Level 3) and the exercise title.
- Objective:** "Show that the change of coordinates between two atlas charts is differentiable to the manifold's order on its domain."
- Active goal:** A list of objects and types defined in the context, including `Scalar`, `Instance`, `Vectors`, `Coordinates`, `model`, `Surface`, `inAtlas`, and `inAtlas'`.
- Available tactics:** A list of tactics available for the proof, including `apply`, `by_cases`, `calc`, `constructor`, `exact`, `ext`, `fun_ext`, `have`, `infer_instance`, `intro`, `left`, `obtain`, `rcases`, `rt`, `right`, `rw`, `simp`, `simp`, and `unfold`.
- World tree:** A tree view showing the current state of the proof, with the current goal highlighted.
- Proof area:** A text area for entering the proof. The current proof is empty.

Ada wants the previous level in plain calculus. Membership in the smooth groupoid was a label; now she unpacks it into the sentence "this function is differentiable `order` times on this set".

`ContDiffOn` is Mathlib's differentiability on a set. The function is the transition map wrapped by the `model`, which converts the abstract coordinate space `Coordinates` into the vector space `Vectors` where derivatives live; for ordinary Euclidean coordinates the model is the identity and can be ignored. The set is the transition's domain, seen through that model. Unpacking uses `mem_groupoid_of_pregroupoid`, an $\leftrightarrow$ statement: membership in the groupoid is equivalent to the conjunction of this condition for the map and for its inverse. An $\leftrightarrow$ has two directions, `.mp` (left to right) and `.mpr` (right to left), and a conjunction's first part is `.1`. The tactic `have` names an intermediate fact so the next line can use it.

Objective: Show that the change of coordinates between two atlas charts is differentiable to the manifold's order on its domain.

GOAL

```
theorem chart_change_contDiffOn {Scalar : Type u} [NontriviallyNormedField Scalar]
  {Vectors : Type v} [NormedAddCommGroup Vectors]
  [NormedSpace Scalar Vectors]
  {Coordinates : Type w} [TopologicalSpace Coordinates]
  {model : ModelWithCorners Scalar Vectors Coordinates}
  {Surface : Type u'} [TopologicalSpace Surface]
  [ChartedSpace Coordinates Surface]
  {order : WithTop ℕ} [IsManifold model order Surface]
  (chart chart' : OpenPartialHomeomorph Surface Coordinates)
  (inAtlas : chart ∈ atlas Coordinates Surface)
  (inAtlas' : chart' ∈ atlas Coordinates Surface) :
  ContDiffOn Scalar order (model ∘ (chart.symm >>_h chart') ∘ model.symm)
    (model.symm -1 (chart.symm >>_h chart').source n Set.range model)
```

SOLUTION

```
have compatible := HasGroupoid.compatible (G := contDiffGroupoid order model) inAtlas
inAtlas'
exact (mem_groupoid_of_pregroupoid.mp compatible).1
```

WHY IT WORKS

Groupoid membership is a pair of differentiability facts, one for the map and one for its inverse. Name the membership with `have`, then take the first component of `mem_groupoid_of_pregroupoid.mp`.

The word "smooth" now means a concrete differentiability statement about a concrete map.

Unlocks: tactics `have`; theorems `mem_groupoid_of_pregroupoid`

Measured: navigation 35 ms, Lean ready 3 ms, goal preview 795 ms, verify 111 ms (kernel 47 ms), accepted by the kernel.

18. Two drawings of the bead agree smoothly

The screenshot shows the Manifold Adventure interface. The sidebar on the left lists levels: 'Smooth manifolds' (Level 4 of 5) and 'Two drawings of the bead agree smoothly'. The main workspace displays the objective: 'Show that the coordinate change between two stereographic projections of the sphere is smooth.' It includes an 'Active goal' section with objects like `Space`, `Instance`, `dimension`, and `pole pole'`. The 'Your proof' section shows a goal with a single subgoal '1'. The right panel contains 'Available tactics' such as `apply`, `by_cases`, `calc`, `constructor`, `exact`, `ext`, `fun_ext`, `have`, `infer_instance`, `intro`, `left`, `obtain`, `rcases`, `rt`, `right`, `rw`, `simp`, `single`, and `unfold`.

Ada returns to the bead with the general fact in hand. Two of her projections, from any two poles, overlap on most of the bead. Reading one drawing into the other must be a smooth map of the plane.

Mathlib proves that the sphere is a smooth manifold, and that instance is enough. The model here is $\mathbb{R}$ dimension, Mathlib's notation for the identity model on `EuclideanSpace  $\mathbb{R}$` (Fin dimension), and ∞ asks for derivatives of every finite order. The general fact `HasGroupoid.compatible` applies once both projections are known to be atlas members, which is the course theorem `sphere_chart_in_atlas` from the atlas world. Coordinates for the projection change are computed by `stereographic'_symm_apply`, but no computation is needed here.

Objective: Show that the coordinate change between two stereographic projections of the sphere is smooth.

Think first: Before proving it: in the plane, the change between the north and south drawings sends a point at distance r from the origin to a point at distance $4/r$ (with Mathlib's scaling). Where does that formula fail, and why does it not matter?

Answer: It fails at $r = 0$, the image of the far pole. That point is missing from the other drawing anyway, so it is not in the overlap where the transition is defined.

GOAL

```
theorem sphere_chart_change_smooth {Space : Type u} [NormedAddCommGroup Space]
  [InnerProductSpace  $\mathbb{R}$  Space] {dimension :  $\mathbb{N}$ }
  [Fact (Module.finiteRank  $\mathbb{R}$  Space = dimension + 1)]
  (pole pole' : sphere (0 : Space) 1) :
  (stereographic' dimension pole).symm  $\gg_n$  stereographic' dimension pole'  $\in$ 
  contDiffGroupoid  $\infty$  ( $\mathbb{R}$  dimension)
```

SOLUTION

```
exact HasGroupoid.compatible (sphere_chart_in_atlas pole) (sphere_chart_in_atlas pole')
```

WHY IT WORKS

The sphere is a smooth manifold, so the general fact about atlas members applies. Feed `HasGroupoid.compatible` the two memberships `sphere_chart_in_atlas pole` and `sphere_chart_in_atlas pole'`.

Ada's two drawings of the bead translate into each other without a crease, which is what makes the bead a smooth manifold.

Measured: navigation 38 ms, Lean ready 2 ms, goal preview 3.3 s, verify 316 ms (kernel 255 ms), accepted by the kernel.

19. Two circles make a torus

The screenshot shows the Manifold Adventure interface for level 5. The main goal is to build the manifold structure on $\text{FirstSurface} \times \text{SecondSurface}$ from its two factors. The interface includes a sidebar with level rewards, a world tree, and a list of smooth manifolds. The main area displays the active goal and a list of available tactics.

Objective: Build the manifold structure on $\text{FirstSurface} \times \text{SecondSurface}$ from its two factors.

Active goal: Initial goal

Objects:

- Scalar :** Type u
- Instance** (NontriviallyNormedField) u (Scalar)
- FirstVectors :** Type v
- Instance** (NormedAddCommGroup) FirstVectors
- Instance** (NormedSpace) Scalar FirstVectors
- SecondVectors :** Type v'
- Instance** (NormedAddCommGroup) SecondVectors
- Instance** (NormedSpace) Scalar SecondVectors
- FirstCoordinates :** Type w
- Instance** (TopologicalSpace) FirstCoordinates
- SecondCoordinates :** Type w'
- Instance** (TopologicalSpace) SecondCoordinates
- FirstModel :** ModelWithCorners Scalar FirstVectors FirstCoordinates
- secondModel :** ModelWithCorners Scalar SecondVectors SecondCoordinates
- FirstSurface :** Type u'
- Instance** (TopologicalSpace) FirstSurface
- Instance** (ChartedSpace) FirstCoordinates FirstSurface
- SecondSurface :** Type u''
- Instance** (TopologicalSpace) SecondSurface
- Instance** (ChartedSpace) SecondCoordinates SecondSurface

Available tactics:

- reply
- by_cases
- calc
- constructor
- exact
- ext
- fun_prop
- have
- infer_instance
- intro
- left
- obtain
- rcases
- refl
- right
- rw
- simp
- simp
- unfold

Verification scope:

Ada's two circular readings describe the torus together. If each circle has smooth coordinate changes, pairing the readings should preserve that smoothness.

Read the final line of the goal first: it asks for a manifold structure on $\text{FirstSurface} \times \text{SecondSurface}$. In Ada's torus, those surfaces are circles. The instance lines above provide their two manifold structures, and Mathlib's `IsManifold.produ` combines them with `firstModel.produ` `secondModel` at the same `order`.

Objective: Build the manifold structure on $\text{FirstSurface} \times \text{SecondSurface}$ from its two factors.

GOAL

```
theorem product_of_manifolds {Scalar : Type u}
  [NontriviallyNormedField Scalar]
  {FirstVectors : Type v}
  [NormedAddCommGroup FirstVectors]
  [NormedSpace Scalar FirstVectors]
  {SecondVectors : Type v'}
  [NormedAddCommGroup SecondVectors]
  [NormedSpace Scalar SecondVectors]
  {FirstCoordinates : Type w}
  [TopologicalSpace FirstCoordinates]
  {SecondCoordinates : Type w'}
  [TopologicalSpace SecondCoordinates]
  {firstModel : ModelWithCorners Scalar FirstVectors FirstCoordinates}
  {secondModel : ModelWithCorners Scalar SecondVectors SecondCoordinates}
  {FirstSurface : Type u'}
  [TopologicalSpace FirstSurface]
  [ChartedSpace FirstCoordinates FirstSurface]
  {SecondSurface : Type u''}
  [TopologicalSpace SecondSurface]
  [ChartedSpace SecondCoordinates SecondSurface]
  (order : WithTop ℕ∞)
  [IsManifold firstModel order FirstSurface]
  [IsManifold secondModel order SecondSurface] :
  IsManifold (firstModel.produ secondModel) order (FirstSurface × SecondSurface)
```

SOLUTION

```
exact IsManifold.prod FirstSurface SecondSurface
```

WHY IT WORKS

The product theorem wants the two surface types. Supply them as `FirstSurface` and `SecondSurface`.

Two smooth circles now give the torus its smooth manifold structure.

Unlocks: theorems `IsManifold.prod`

Measured: navigation 37 ms, Lean ready 2 ms, goal preview 792 ms, verify 115 ms (kernel 43 ms), accepted by the kernel.

A direction at every point

Manifold Adventure

WIP Work in progress

Games

Worlds

Lean playground

Game source

Back to world tree

LOCAL KERNEL

A direction at every point

0/5 complete

A direction at every point

Ada's atlas tells her where she is. At one point on the surface, she now asks which directions she could move without leaving it. On the bead the answer is visible: a plane touching the bead at her feet, square to the radius. The world builds that picture as theorems.

Mathlib assigns a `TangentSpace` to every point, and the derivative of a smooth map between manifolds, `mfdderiv`, carries velocities from one tangent space to another. In the goals, `Surface` is Ada's world, `place` is her location, `model` describes its coordinates, and `velocity` is a tangent vector there. Drag the point in the tangent lab below to see the plane follow it around the bead.

Tangent lab

Tangent plane on the bead

Direction of the walk34°

Stand on the north pole

Stand on the equator

01Ada stands stillLocal kernel

02Read the location tagLocal kernel

03Leaving the bead for the room around itLocal kernel

04Her velocities are real vectorsLocal kernel

05The tangent plane stands square to the radiusLocal kernel

A direction at every point

Ada's atlas tells her where she is. At one point on the surface, she now asks which directions she could move without leaving it. On the bead the answer is visible: a plane touching the bead at her feet, square to the radius. The world builds that picture as theorems.

Mathlib assigns a `TangentSpace` to every point, and the derivative of a smooth map between manifolds, `mfdderiv`, carries velocities from one tangent space to another. In the goals, `Surface` is Ada's world, `place` is her location, `model` describes its coordinates, and `velocity` is a tangent vector there. Drag the point in the tangent lab below to see the plane follow it around the bead.

Measured: world page opened in 219 ms.

The Manifold Adventure r5 · page 39 of 75

20. Ada stands still

Ada stands still

Ada stands still at `place`. Even without choosing a direction, staying still is a valid tangent velocity.

The `TangentSpace` `TangentSpace model place` is the intrinsic space of velocities available at Ada's current location. The plane in the 3D scene pictures this tangent space; it is not an arbitrary plane floating beside the surface. The space inherits an additive group structure from `Vectors`, so it contains a zero vector. The expected type tells Lean which `0` is intended.

This is a definition level, so the kernel accepts any well-typed term. Only one velocity here is canonical, and later levels reuse the course's official `tangent_zero`, so make it the zero vector.

Objective: Construct the zero tangent vector at `place`.

Active goal

Objects

- `Scalar` : Type `u`
- `Instance` `(NontriviallyNormedField Id Scalar)`
- `Vectors` : Type `v`
- `Instance` `(NormedAddCommGroup Vectors)`
- `Instance` `(NormedSpace Scalar Vectors)`
- `Coordinates` : Type `w`
- `Instance` `(TopologicalSpace Coordinates)`
- `model` : `ModelWithCorners Scalar Vectors Coordinates`
- `Surface` : Type `u'`
- `Instance` `(TopologicalSpace Surface)`
- `Instance` `(ChartedSpace Coordinates Surface)`
- `place` : `Surface`

Your proof

Enter tactics only. The surrounding `by` is supplied for you. 0 attempts

`by 1`

Available tactics

apply | by_cases | calc | constructor
exact | ext | fun_ext | have
infer_instance | intro | left | obtain
rcases | rfl | right | rw | simp
simp | unfold |

► Verification scope

Ada stands still at `place`. Even without choosing a direction, staying still is a valid tangent velocity.

The `TangentSpace` `TangentSpace model place` is the intrinsic space of velocities available at Ada's current location. The plane in the 3D scene pictures this tangent space; it is not an arbitrary plane floating beside the surface. The space inherits an additive group structure from `Vectors`, so it contains a zero vector. The expected type tells Lean which `0` is intended.

This is a definition level, so the kernel accepts any well-typed term. Only one velocity here is canonical, and later levels reuse the course's official `tangent_zero`, so make it the zero vector.

The `TangentBundle` collects each place together with one of its velocities as a dependent pair `{place, velocity}`. This level constructs one velocity at one point; it does not yet define the zero section as a function of the point.

Objective: Construct the zero tangent vector at `place`.

GOAL

```
noncomputable def tangent_zero {Scalar : Type u}
  [NontriviallyNormedField Scalar]
  {Vectors : Type v} [NormedAddCommGroup Vectors]
  [NormedSpace Scalar Vectors]
  {Coordinates : Type w}
  [TopologicalSpace Coordinates]
  (model : ModelWithCorners Scalar Vectors Coordinates)
  {Surface : Type u'} [TopologicalSpace Surface]
  [ChartedSpace Coordinates Surface] (place : Surface) :
  TangentSpace model place
```

SOLUTION

```
exact 0
```

WHY IT WORKS

The tangent space has a zero instance. The expected type is enough for Lean to understand `0`.

Standing still is now a genuine vector in `TangentSpace model place`.

Measured: navigation 252 ms, Lean ready 16 ms, goal preview 798 ms, verify 102 ms (kernel 29 ms), accepted by the kernel.

21. Read the location tag

Objective: Show that projecting the base point from $(\text{place}, \text{velocity})$ returns place .

Active goal

Objects

- Scalar** : Type u
 Instance (NontriviallyNormedField u Scalar)
- Vectors** : Type v
 Instance (NormedAddCommGroup Vectors)
 Instance (NormedSpace Scalar Vectors)
- Coordinates** : Type w
 Instance (TopologicalSpace Coordinates)
- model** : ModelWithCorners Scalar Vectors Coordinates
- Surface** : Type u'
 Instance (TopologicalSpace Surface)
 Instance (ChartedSpace Coordinates Surface)
- place** : Surface
velocity : TangentSpace model place

Your proof

Error tactic only. The surrounding by is supplied for you. # attempts

by 1

Available tactics

apply | by_cases | calc | constructor
 exact | ext | fun_arg | have
 infer_instance | intro | left | obtain
 rcases | rfl | right | rw | simp
 simp | unfold

Verification scope

Ada records both where she is and the direction she is moving, then reads the record's location tag. A direction without its point would be ambiguous because the available tangent plane changes from place to place. The tag must give back the point she stored.

A point of the `TangentBundle` is the dependent pair $(\text{place}, \text{velocity})$ (angle brackets: type $\lambda<$ and $\lambda>$). The tangent space may change with `place`, so `velocity : TangentSpace model place` remembers where the velocity belongs. The first projection `.1` reduces by definition to `place`, and the `rfl` tactic checks that reduction.

Objective: Show that projecting the base point from $(\text{place}, \text{velocity})$ returns place .

GOAL

```
theorem tangent_bundle_base {Scalar : Type u}
  [NontriviallyNormedField Scalar]
  {Vectors : Type v} [NormedAddCommGroup Vectors]
  [NormedSpace Scalar Vectors]
  {Coordinates : Type w}
  [TopologicalSpace Coordinates]
  (model : ModelWithCorners Scalar Vectors Coordinates)
  {Surface : Type u'} [TopologicalSpace Surface]
  [ChartedSpace Coordinates Surface]
  {place : Surface} (velocity : TangentSpace model place) :
  ((place, velocity) : TangentBundle model Surface).1 = place
```

SOLUTION

```
rfl
```

WHY IT WORKS

The first projection reduces to `place` by definition. A reflexivity proof closes such a goal.

Reading the bundle point's location tag returns `place`.

Measured: navigation 261 ms, Lean ready 6 ms, goal preview 809 ms, verify 93 ms (kernel 27 ms), accepted by the kernel.

22. Leaving the bead for the room around it

The screenshot shows the Manifold Adventure interface. On the left, a sidebar contains a 'World tree' with a list of levels: 'One trail, one leaf' (4/4), 'One pole is missing' (5/5), 'A stack of maps' (5/5), 'Smooth manifolds' (5/5), 'A direction at every point' (2/5), and 'Leaving the bead for the room around it' (1/5). The main area displays the 'Objective: Show that the inclusion of the sphere into the surrounding space is smooth as a map between manifolds.' Below this, the 'Active goal' section shows the Lean code for the theorem. The 'Your proof' section shows a single tactic 'by 1'. On the right, a panel lists 'Available tactics' such as 'reply', 'by_cases', 'calc', 'constructor', 'exact', 'ext', 'fun_arg', 'have', 'infer_instance', 'intro', 'left', 'obtain', 'rcases', 'rfl', 'right', 'rw', 'simp', 'skip', and 'unfold'.

Ada has only ever measured positions on the bead. The bead also sits in the room, and each of her positions is a point of the room. Forgetting the bead and remembering only the room point should be a smooth operation.

`ContMDiff` is smoothness for a map between two manifolds, each with its own model: $\mathcal{R}$ dimension for the sphere and $\mathcal{I}(\mathbb{R}, \text{Space})$, the identity model, for the surrounding vector space. The map $(\uparrow)$ is the inclusion that forgets the constraint $\|place\| = 1$. Mathlib records its smoothness as `contMDiff_coe_sphere`.

Objective: Show that the inclusion of the sphere into the surrounding space is smooth as a map between manifolds.

GOAL

```
theorem sphere_inclusion_smooth {Space : Type u} [NormedAddCommGroup Space]
  [InnerProductSpace ℝ Space] {dimension : ℕ}
  [Fact (Module.finite ℝ Space = dimension + 1)] :
  ContMDiff (ℝ dimension)  $\mathcal{I}(\mathbb{R}, \text{Space})$   $\infty$  ((↑) : sphere (0 : Space) 1 → Space)
```

SOLUTION

```
exact contMDiff_coe_sphere
```

WHY IT WORKS

Mathlib proves that the inclusion of the sphere is a smooth map of manifolds. The theorem is `contMDiff_coe_sphere`.

Ada's first smooth map between manifolds sends the bead into the room without a kink.

Unlocks: theorems `contMDiff_coe_sphere`

Measured: navigation 688 ms, Lean ready 11 ms, goal preview 1.3 s, verify 228 ms (kernel 90 ms), accepted by the kernel.

23. Her velocities are real vectors

Objective: Show that the derivative of the inclusion at `pplace` is injective, so tangent vectors of the sphere are genuine vectors of the surrounding space.

Active goal

```

Space : Type u
Instance (NormedAddCommGroup Space)
Instance (InnerProductSpace ℝ Space)
dimension : ℕ
place : sphere (0 : Space) 1
Assumptions
Instance (Fact (Module.finite ℝ Space = dimension + 1))
Function.Injective
  (mderiv (ℝ dimension)
    J(ℝ, Space) ((↑) : sphere (0 : Space) 1 → Space) place)

```

Your proof

Enter tactics only. The surrounding `by` is supplied for you.

```

by 1

```

Available tactics

reply by_cases calc constructor
 exact ext fun_arg have
 infer_instance intro left obtain
 rcases rfl right rw simp
 simp unfold

► Verification scope

Ada picks a velocity at her place on the bead. Since the bead sits in the room, that velocity is also a velocity in the room. Two different velocities on the bead must stay different in the room; nothing is flattened away.

The derivative of a smooth map between manifolds is `mderiv`. At `place`, it is a linear map from `TangentSpace (ℝ dimension) place` to the tangent space of the room, which is the room itself. `Function.Injective` says this linear map loses nothing, and `mderiv_coe_sphere_injective` proves it for the inclusion of the sphere.

Objective: Show that the derivative of the inclusion at `pplace` is injective, so tangent vectors of the sphere are genuine vectors of the surrounding space.

GOAL

```

theorem tangent_vectors_are_vectors {Space : Type u} [NormedAddCommGroup Space]
  [InnerProductSpace ℝ Space] {dimension : ℕ}
  [Fact (Module.finite ℝ Space = dimension + 1)]
  (place : sphere (0 : Space) 1) :
  Function.Injective
    (mderiv (ℝ dimension) J(ℝ, Space) ((↑) : sphere (0 : Space) 1 → Space) place)

```

SOLUTION

```

exact mderiv_coe_sphere_injective place

```

WHY IT WORKS

The derivative of the inclusion embeds each tangent space into the room. The theorem is `mderiv_coe_sphere_injective` applied to `place`.

Every abstract tangent vector at Ada's place is a real vector in the room, and different ones stay different.

Unlocks: theorems `mderiv_coe_sphere_injective`

Measured: navigation 750 ms, Lean ready 70 ms, goal preview 799 ms, verify 232 ms (kernel 110 ms), accepted by the kernel.

24. The tangent plane stands square to the radius

The screenshot shows the Manifold Adventure interface. On the left, a sidebar contains a 'World tree' with a list of goals: 'One ball, one leaf' (4/4), 'One pole is missing' (5/5), 'A stack of maps' (5/5), 'Smooth manifolds' (5/5), and 'A direction at every point' (4/5). The 'A direction at every point' goal is expanded, showing a list of sub-goals: 'Ada stands still' (✓), 'Read the location tag' (✓), and 'Locate the bead for the room around it' (✓). The main area displays the 'Objective: Show that the tangent vectors at `place`, seen in the room, are exactly the vectors orthogonal to `place`.' Below this, the 'Active goal' section shows the 'Initial goal' and the 'Your proof' section with a 'by' line. The 'Available tactics' section on the right lists various tactics: `apply`, `by_cases`, `calc`, `constructor`, `exact`, `ext`, `fun_ext`, `have`, `infer_instance`, `intro`, `left`, `obtain`, `rcases`, `refl`, `right`, `rw`, `simp`, `single`, and `unfold`.

Ada asks which room vectors her bead velocities become. She expects exactly the vectors that lie flat against the bead at her place: perpendicular to the line from the center through her feet.

The image of the derivative is `LinearMap.range`; the derivative is a continuous linear map, and `ContinuousLinearMap.toLinearMap` forgets the continuity so that `range` applies. The line through `place` is the span $\mathbb{R} \cdot (\text{place} : \text{Space})$, and $\perp$ (type `\perp`) is its `Submodule.orthogonal` complement. Mathlib's `range_mfderiv_coe_sphere` identifies the two. This is the tangent plane drawn in the scene, now as a theorem.

Objective: Show that the tangent vectors at `place`, seen in the room, are exactly the vectors orthogonal to `place`.

Think first: Before proving it: the sphere in a space of dimension $d+1$ has tangent spaces of which dimension, and how does the orthogonal complement of one vector confirm it?

Answer: Dimension d . The orthogonal complement of a single nonzero vector in a $(d+1)$ -dimensional space has dimension d .

GOAL

```
theorem tangent_plane_is_orthogonal {Space : Type u} [NormedAddCommGroup Space]
  [InnerProductSpace ℝ Space] {dimension : ℕ}
  [Fact (Module.finite ℝ Space = dimension + 1)]
  (place : sphere (0 : Space) 1) :
  LinearMap.range (ContinuousLinearMap.toLinearMap
    (mfderiv (ℝ dimension) J(ℝ, Space) (↑ : sphere (0 : Space) 1 → Space) place))
  = (ℝ • (place : Space))⊥
```

SOLUTION

```
exact range_mfderiv_coe_sphere place
```

WHY IT WORKS

The tangent plane of a sphere is perpendicular to the radius, and Mathlib says so about the range of the derivative. The theorem is `range_mfderiv_coe_sphere` applied to `place`.

The tangent plane at Ada's place is the plane perpendicular to her radius, exactly as the picture suggested.

Unlocks: theorems `range_mfderiv_coe_sphere`

Measured: navigation 805 ms, Lean ready 14 ms, goal preview 1.8 s, verify 336 ms (kernel 191 ms), accepted by the kernel.

Identity and product charts

Manifold Adventure

WIP · Work in progress

Games

Worlds

Lean playground

Game source

Back to world tree

LOCAL KERNEL

Identity and product charts

0/5 complete

Charts Lean already knows

Ada sets two identical reference grids on top of each other before returning to the curved surface. One maps to the other without moving a mark. A place with two independent readings needs a pair of maps.

Mathlib supplies canonical `ChartedSpace` instances for self charts and products. Here the goal names the two torus factors `FirstSurface` and `SecondSurface`, together with their coordinate spaces. The types determine which instance Lean uses, even though the notation `chartAt` stays the same. The shape gallery below is optional; some objects return in the levels, while others preview later topology.

OPTIONAL TOPOLOGY GALLERY

Sphere with two charts

Sphere with two charts

Torus with its two loops

01

The reference grid stays put

Local kernel

02

The identity is filed in the atlas

Local kernel

03

Only the identity is filed there

Local kernel

04

Two readings at once

Local kernel

05

The paired chart contains her place

Local kernel

Charts Lean already knows

Ada sets two identical reference grids on top of each other before returning to the curved surface. One maps to the other without moving a mark. A place with two independent readings needs a pair of maps.

Mathlib supplies canonical `ChartedSpace` instances for self charts and products. Here the goal names the two torus factors `FirstSurface` and `SecondSurface`, together with their coordinate spaces. The types determine which instance Lean uses, even though the notation `chartAt` stays the same. The shape gallery below is optional; some objects return in the levels, while others preview later topology.

Measured: world page opened in 19 ms.

The Manifold Adventure r5 · page 47 of 75

25. The reference grid stays put

Objective: Show that a space used as its own coordinate model has the identity as its preferred chart.

Active goal

Objects

- `Coordinates` : Type `u`
- Instance** `(TopologicalSpace Coordinates)` : `chartAt Coordinates mark = OpenPartialHomeomorph.refl Coordinates`
- mark** : `Coordinates`

Your proof

Enter tactics only. The surrounding `by` is supplied for you.

by 1

Available tactics

reply by_cases calc constructor exact ext fun_arg have infer_instance intro left obtain rcases rfl right rw simp simp unfold

► Verification scope

World tree	Progress
One ball, one leaf	4/4
One pole is missing	5/5
A stack of maps	5/5
Smooth manifolds	5/5
A direction at every point	5/5
Identity and product charts	8/8

Ada lays one reference grid on top of an identical grid. Every mark already sits in the right place, so the map does nothing.

Both grids are represented by the same type, `Coordinates`, and `mark` is one point on them. Mathlib's canonical `ChartedSpace Coordinates Coordinates` instance uses `OpenPartialHomeomorph.refl Coordinates`. The theorem `chartAt_self_eq` describes this chosen self-chart; it does not say that every atlas on `Coordinates` must use only identity charts.

Objective: Show that a space used as its own coordinate model has the identity as its preferred chart.

GOAL

```
theorem self_chart_is_identity {Coordinates : Type u}
  [TopologicalSpace Coordinates]
  (mark : Coordinates) :
    chartAt Coordinates mark = OpenPartialHomeomorph.refl Coordinates
```

SOLUTION

```
exact chartAt_self_eq
```

WHY IT WORKS

The canonical self-charted instance uses one chart. Use `chartAt_self_eq`; all arguments are implicit.

The reference leaf needs only the identity chart.

Unlocks: theorems `chartAt_self_eq`

Measured: navigation 75 ms, Lean ready 2 ms, goal preview 795 ms, verify 62 ms (kernel 16 ms), accepted by the kernel.

26. The identity is filed in the atlas

Ada opens the small atlas that came with the reference grid and files the identity chart into it. Matching the grid with itself is the only map this atlas was meant to hold.

Mathlib's `chartedSpaceSelf_atlas` is an $\Leftrightarrow$: a chart belongs to `atlas Coordinates Coordinates` exactly when it is the identity. Its two directions are `.mp` (left to right) and `.mpr` (right to left). This membership goal uses the right-to-left direction, fed with `rfl`, a proof that the identity equals itself.

Objective: Show that the identity chart belongs to the self-atlas.

GOAL

```
theorem identity_mem_self_atlas {Coordinates : Type u}
  [TopologicalSpace Coordinates] :
    OpenPartialHomeomorph.refl Coordinates ∈ atlas Coordinates Coordinates
```

SOLUTION

```
exact chartedSpaceSelf_atlas.mpr rfl
```

WHY IT WORKS

Read the atlas-membership equivalence backwards. Use `chartedSpaceSelf_atlas.mpr`; it wants the equality proof `rfl`.

The reference atlas accepts its one and only chart.

Unlocks: theorems `chartedSpaceSelf_atlas`

Measured: navigation 34 ms, Lean ready 1 ms, goal preview 791 ms, verify 58 ms (kernel 16 ms), accepted by the kernel.

27. Only the identity is filed there

Identity and product charts · Level 3 of 5 · Only the identity is filed there

Objective: From atlas membership, conclude that the chart is the identity.

Active goal: Initial goal

Objects:

- Coordinates** : Type u
- Instance** (TopologicalSpace Coordinates)
- chart** : OpenPartialHomeomorph Coordinates Coordinates
- inAtlas** : chart ∈ atlas Coordinates Coordinates

chart = OpenPartialHomeomorph.refl Coordinates

Your proof

Enter tactics only. The surrounding `by` is supplied for you. # attempts

by 1

Available tactics: apply, by_cases, calc, constructor, exact, ext, fun_ext, have, infer_instance, intro, left, obtain, rcases, rfl, right, rw, simp, simp_unfold, Verification scope

Level rewards: New file best (177 ms), Prove the course declaration (self_atlas_chart_is_identity), Maths names are binary declarations; the course declaration is your reusable result.

World tree:

- One trail, one leaf 4/4
- One pole is missing 5/5
- A stack of maps 5/5
- Smooth manifolds 5/5
- A direction at every point 5/5
- Identity and product charts 2/5
- 1. The reference grid stops put 1/1

Verify answer Show a hint View solution

Ada pulls a chart out of the reference atlas. Whatever leaf she is holding, the atlas accepted only one map, so it must be the identity.

This is the forward direction of `chartedSpaceSelf_atlas`. Its `.mp` projection turns the membership hypothesis `inAtlas` into the required equality.

Objective: From atlas membership, conclude that the chart is the identity.

GOAL

```
theorem self_atlas_chart_is_identity {Coordinates : Type u}
  [TopologicalSpace Coordinates]
  (chart : OpenPartialHomeomorph Coordinates Coordinates)
  (inAtlas : chart ∈ atlas Coordinates Coordinates) :
  chart = OpenPartialHomeomorph.refl Coordinates
```

SOLUTION

```
exact chartedSpaceSelf_atlas.mp inAtlas
```

WHY IT WORKS

Read the same equivalence forwards this time. Use `.mp` to turn `inAtlas` into the equality.

Every chart the reference atlas hands Ada is the identity.

Measured: navigation 35 ms, Lean ready 2 ms, goal preview 791 ms, verify 64 ms (kernel 18 ms), accepted by the kernel.

28. Two readings at once

Objective: Show that the preferred chart of a paired point is the product of its two component charts.

Active goal

Objects

- `FirstCoordinates : Type u`
- `SecondCoordinates : Type u'`
- `FirstSurface : Type v`
- `SecondSurface : Type v'`

Instances

- `(TopologicalSpace FirstCoordinates)`
- `(TopologicalSpace SecondCoordinates)`
- `(TopologicalSpace FirstSurface)`
- `(TopologicalSpace SecondSurface)`
- `(ChartedSpace FirstCoordinates FirstSurface)`
- `(ChartedSpace SecondCoordinates SecondSurface)`

position : FirstSurface × SecondSurface

chartAt (ModelProd FirstCoordinates SecondCoordinates) position = (chartAt FirstCoordinates position.1).prod (chartAt SecondCoordinates position.2)

Your proof

Enter tactics only. The surrounding `by` is supplied for you.

`by 1`

On a torus, Ada records two positions at once: how far she has gone around the hole and how far she has gone around the tube. Each reading has its own local map.

Think first of `FirstSurface` and `SecondSurface` as two circles whose product is a torus. The two entries of `position : FirstSurface × SecondSurface` are Ada's two readings. Mathlib combines their coordinate types as `ModelProd FirstCoordinates SecondCoordinates`. The theorem `prodChartedSpace_chartAt` says that the preferred chart is the product of the two component charts.

Objective: Show that the preferred chart of a paired point is the product of its two component charts.

GOAL

```
theorem product_chart_is_product {FirstCoordinates : Type u} {SecondCoordinates : Type u'}
  {FirstSurface : Type v} {SecondSurface : Type v'}
  [TopologicalSpace FirstCoordinates]
  [TopologicalSpace SecondCoordinates]
  [TopologicalSpace FirstSurface]
  [TopologicalSpace SecondSurface]
  [ChartedSpace FirstCoordinates FirstSurface]
  [ChartedSpace SecondCoordinates SecondSurface]
  (position : FirstSurface × SecondSurface) :
  chartAt (ModelProd FirstCoordinates SecondCoordinates) position =
    (chartAt FirstCoordinates position.1).prod
    (chartAt SecondCoordinates position.2)
```

SOLUTION

```
rw [prodChartedSpace_chartAt]
```

WHY IT WORKS

The product instance computes its chart by a stated rule. Rewrite with `prodChartedSpace_chartAt`.

The torus chart is built by reading its two coordinates side by side.

Unlocks: tactics `rw`; theorems `prodChartedSpace_chartAt`

Measured: navigation 248 ms, Lean ready 35 ms, goal preview 847 ms, verify 180 ms (kernel 30 ms), accepted by the kernel.

29. The paired chart contains her place

Ada combines one position from each loop of the torus. The paired point must lie inside the source of the paired chart.

The pair `(firstPosition, secondPosition)` records Ada's place in both factors. The earlier theorem `mem_chart_source` also applies to the product charted-space instance, which Lean infers from `ModelProd FirstCoordinates SecondCoordinates`. Here `exact` needs help because `ModelProd` is a type synonym. The tactic `simp` only using `h` unfolds just enough notation in the goal and `h` to make them match. Its cousin `simp` uses Mathlib's default simplification lemmas.

Objective: Show that the paired position lies inside its preferred product chart.

GOAL

```
theorem product_point_mem_chart_source {FirstCoordinates : Type u} {SecondCoordinates : Type u'}
  {FirstSurface : Type v} {SecondSurface : Type v'}
  [TopologicalSpace FirstCoordinates]
  [TopologicalSpace SecondCoordinates]
  [TopologicalSpace FirstSurface]
  [TopologicalSpace SecondSurface]
  [ChartedSpace FirstCoordinates FirstSurface]
  [ChartedSpace SecondCoordinates SecondSurface]
  (firstPosition : FirstSurface) (secondPosition : SecondSurface) :
  (firstPosition, secondPosition) ∈
    (chartAt (ModelProd FirstCoordinates SecondCoordinates)
      (firstPosition, secondPosition)).source
```

SOLUTION

```
simp only using
  (mem_chart_source (ModelProd FirstCoordinates SecondCoordinates)
    (firstPosition, secondPosition))
```

WHY IT WORKS

Specialize the earlier covering theorem to the product model. Then use `simp only` with the paired position.

The paired chart contains the paired point, just as each component chart contains its own point.

Unlocks: tactics `simp`, `simp`

Measured: navigation 128 ms, Lean ready 2 ms, goal preview 851 ms, verify 179 ms (kernel 29 ms), accepted by the kernel.

How smooth is smooth

Manifold Adventure

WIP | Work in progress

Games

Worlds

Lean playground

Game source

Back to world tree

LOCAL KERNEL

How smooth is smooth

0/3 complete

Counting derivatives

Ada checks her map changes against standards of different strictness. A leaf change that passes a demanding test passes every easier one, and the model leaf passes them all.

These three levels spell out how Mathlib's `IsManifold` orders relate: the model space is a manifold at every order, a higher order implies a lower one, and a smooth atlas is in particular a topological one. The order `0` here is a regularity order, not a dimension.

01

The reference leaf is ready

Local kernel

02

Passing an easier check

Local kernel

03

The smooth atlas passes the basic check

Local kernel

Convenience port of Lean4Game; upstream creators and licenses retain credit.

Mathlib proof checking stays in this browser.

Counting derivatives

Ada checks her map changes against standards of different strictness. A leaf change that passes a demanding test passes every easier one, and the model leaf passes them all.

These three levels spell out how Mathlib's `IsManifold` orders relate: the model space is a manifold at every order, a higher order implies a lower one, and a smooth atlas is in particular a topological one. The order `0` here is a regularity order, not a dimension.

Measured: world page opened in 27 ms.

The Manifold Adventure r5 · page 54 of 75

30. The reference leaf is ready

Manifold Adventure
Workshop in progress
The reference leaf... (not)
The reference leaf is ready

How smooth is smooth
/ Level 1+2+3

LOCAL KERNEL

The reference leaf is ready

Ada places a model leaf beside the world she is charting. The leaf is already its own coordinate space, so it needs no further change of coordinates to qualify as a manifold.

In the good `Scalar`, supplies the numbers.
`Vectors`: supplies directions, and
`Coordinates`: is the model leaf itself. The `ModelWithCurviers` named `model` connects those pieces. The ordered type `WithTop /w` records differentiability levels. Here `#` means continuity-level regularity, `w` means smoothness at every finite order, and the top element `i∞` means analytic. The optional circle word meets that stronger standard. Mathlib registers `instOfmanifoldModelSpace` for every

World tree	
One ball, one leaf	4/4
One pole is missing	5/5
A stack of maps	5/5
Smooth manifolds	5/5
A direction at every point	5/5
Identify and product charts	5/5
How smooth is smooth	8/3

Objective: Establish that `(Coordinates) carries the manifold structure supplied by model`.

```

-- | Manifold model order Coordinates
Active goal                                     Initial goal

Objectives
-----
Scalar : Type u
instance (MetricRiemannianId id Scalar)

Vectors : Type v
instance (NormedAddCommGroup Vectors)

Coordinates : Type w
instance (TopologicalSpace Coordinates)

model : ModelWithCurviers Scalar
       Vectors.Coordinates
order : WithTop Ww
    
```

Enter tactics only! The surrounding `y` is supplied for you.

```
by 1
```

TACTICS THEOREMS DEFINITIONS

Available tactics

apply	by_cases	cac!	constructor
exact	ext	fsm_step	have
infer_instance	intro	left	obtain
rcases	rfl	right	rw
simp	unfold	-	xsimp

▼ Refinement scope

Ada places a model leaf beside the world she is charting. The leaf is already its own coordinate space, so it needs no further change of coordinates to qualify as a manifold.

In the goal, `Scalar` supplies the numbers, `Vectors` supplies directions, and `Coordinates` is the model leaf itself. The `ModelWithCorners` named `model` connects those pieces. The ordered type `WithTop  $\mathbb{N}_\infty$` records differentiability levels. Here `0` means continuity-level regularity, `$\infty$` means smoothness at every finite order, and the top element `$\omega$` means analyticity. The optional circle world meets that stronger standard. Mathlib registers `instIsManifoldModelSpace` for every order.

Objective: Establish that `Coordinates` carries the manifold structure supplied by `model`.

GOAL

```
theorem model_space_is_manifold {Scalar : Type u}
  [NontriviallyNormedField Scalar]
  {Vectors : Type v} [NormedAddCommGroup Vectors]
  [NormedSpace Scalar Vectors]
  {Coordinates : Type w}
  [TopologicalSpace Coordinates]
  (model : ModelWithCorners Scalar Vectors Coordinates)
  (order : WithTop  $\mathbb{N}_\infty$ ) :
  IsManifold model order Coordinates
```

SOLUTION

```
infer instance
```

WHY IT WORKS

Mathlib has registered this result as an instance. Ask typeclass inference to find it.

The model space is already a manifold at the requested order.

Unlocks: tactics `infer_instance`; theorems `instIsManifoldModelSpace`

Measured: navigation 36 ms, Lean ready 2 ms, goal preview 794 ms, verify 114 ms (kernel 50 ms), accepted by the kernel.

31. Passing an easier check

How smooth is smooth · Level 2 of 3

Passing an easier check

Ada checks her map changes to a demanding standard. If they pass that test, they also pass any test that asks for fewer derivatives.

For example, an atlas of class C^5 also meets a C^2 requirement. Lean calls the demanding standard `higherOrder` and the weaker one `lowerOrder`. This time the comparison arrives inside the goal as an implication. The `intro` tactic moves its assumption into the context. Mathlib's `IsManifold.of_le` then finishes.

Level rewards

World tree

One ball, one leaf 4/4

One pole is missing 5/5

A stack of maps 5/5

Smooth manifolds 5/5

A direction at every point 5/5

Identify and product charts 5/5

How smooth is smooth 1/3

Objective: Assuming `lowerOrder ≤ higherOrder`, lower the known differentiability order from `higherOrder` to `lowerOrder`.

Active goal

Initial goal

Objects

Scalar : Type u

Instance (NontriviallyNormedField Scalar)

Vectors : Type v

Instance (NormedAddCommGroup Vectors)

Instance (NormedSpace Scalar Vectors)

Coordinates : Type w

Instance (TopologicalSpace Coordinates)

model : ModelWithCorners Scalar Vectors Coordinates

Surface : Type u

Instance (TopologicalSpace Surface)

Instance (CharteredSpace Coordinates Surface)

lowerOrder higherOrder : WithTop $\mathbb{N}_\infty$

Instance (IsManifold model higherOrder Surface)

lowerOrder ≤ higherOrder → IsManifold model lowerOrder Surface

Your proof

Enter tactics only. The surrounding `by` is supplied for you.

by 1

Available tactics

reply by_cases calc constructor

exact ext fun_arg have

infer_instance intro left obtain

rcases rfl right rw simp

simp unfold

Verification scope

Ada checks her map changes to a demanding standard. If they pass that test, they also pass any test that asks for fewer derivatives.

For example, an atlas of class C^5 also meets a C^2 requirement. Lean calls the demanding standard `higherOrder` and the weaker one `lowerOrder`. This time the comparison arrives inside the goal as an implication. The `intro` tactic moves its assumption into the context. Mathlib's `IsManifold.of_le` then finishes.

Objective: Assuming `lowerOrder ≤ higherOrder`, lower the known differentiability order from `higherOrder` to `lowerOrder`.

GOAL

```
theorem manifold_of_higher_smoothness {Scalar : Type u}
  [NontriviallyNormedField Scalar]
  {Vectors : Type v} [NormedAddCommGroup Vectors]
  [NormedSpace Scalar Vectors]
  {Coordinates : Type w}
  [TopologicalSpace Coordinates]
  {model : ModelWithCorners Scalar Vectors Coordinates}
  {Surface : Type u} [TopologicalSpace Surface]
  [CharteredSpace Coordinates Surface]
  {lowerOrder higherOrder : WithTop  $\mathbb{N}_\infty$ }
  [IsManifold model higherOrder Surface] :
  lowerOrder ≤ higherOrder → IsManifold model lowerOrder Surface
```

SOLUTION

```
intro order_le
exact IsManifold.of_le order_le
```

WHY IT WORKS

Bring the implication's assumption into the context first. After `intro order_le`, pass it to `IsManifold.of_le`.

The higher-order manifold instance now works at the requested lower order.

Unlocks: tactics `intro`; theorems `IsManifold.of_le`

Measured: navigation 52 ms, Lean ready 1 ms, goal preview 796 ms, verify 113 ms (kernel 26 ms), accepted by the kernel.

32. The smooth atlas passes the basic check

How smooth is smooth · Level 3 of 3

The smooth atlas passes the basic check

Ada's smoothest leaf changes never crease or kink. They certainly still preserve the nearby-point structure she needed for her first maps.

The assumption `IsManifold model ∞ Surface` says that Ada's chart changes have derivatives of every finite order. Mathlib's `IsManifold` hierarchy registers the implication to `IsManifold model 0 Surface`, where order `0` retains the basic topological requirement. This `0` is a regularity order, not the dimension of `Surface`.

Level rewards

World tree	Rewards
One ball, one leaf	4/4
One pole is missing	5/5
A stack of maps	5/5
Smooth manifolds	5/5
A direction at every point	5/5
Identity and product charts	5/5
How smooth is smooth	2/3

Objective: Derive the topological manifold structure from the smooth one.

Active goal

```

Scalar : Type u
Instance (NontriviallyNormedField Scalar)
Vectors : Type v
Instance (NormedAddCommGroup Vectors)
Instance (NormedSpace Scalar Vectors)
Coordinates : Type w
Instance (TopologicalSpace Coordinates)
model : ModelWithCorners Scalar Vectors Coordinates
Surface : Type u'
Instance (TopologicalSpace Surface)
Instance (ChartedSpace Coordinates Surface)
Instance (IsManifold model ∞ Surface)

```

Initial goal

```

IsManifold model 0 Surface

```

Available tactics

apply | by_cases | calc | constructor
 exact | ext | fun_ext | have
 infer_instance | intro | left | obtain
 rcases | rfl | right | rw | simp
 skip | unfold |

Verification scope

Your proof

Enter tactics only. The surrounding `by` is supplied for you. # attempts

```

by 1

```

Ada's smoothest leaf changes never crease or kink. They certainly still preserve the nearby-point structure she needed for her first maps.

The assumption `IsManifold model ∞ Surface` says that Ada's chart changes have derivatives of every finite order. Mathlib's `IsManifold` hierarchy registers the implication to `IsManifold model 0 Surface`, where order `0` retains the basic topological requirement. This `0` is a regularity order, not the dimension of `Surface`.

Objective: Derive the topological manifold structure from the smooth one.

GOAL

```

theorem smooth_manifold_is_topological {Scalar : Type u}
  [NontriviallyNormedField Scalar]
  {Vectors : Type v} [NormedAddCommGroup Vectors]
  [NormedSpace Scalar Vectors]
  {Coordinates : Type w}
  [TopologicalSpace Coordinates]
  {model : ModelWithCorners Scalar Vectors Coordinates}
  {Surface : Type u'} [TopologicalSpace Surface]
  [ChartedSpace Coordinates Surface]
  [IsManifold model ∞ Surface] :
  IsManifold model 0 Surface

```

SOLUTION

```
infer_instance
```

WHY IT WORKS

Mathlib registers this implication as an instance. Let `infer_instance` find it.

The smooth atlas also gives Ada the topological atlas she started with.

Measured: navigation 34 ms, Lean ready 1 ms, goal preview 792 ms, verify 113 ms (kernel 29 ms), accepted by the kernel.

The dial comes around

Manifold Adventure

WIP | Work in progress

Games

Worlds

Lean playground

Game source

Back to world tree

LOCAL KERNEL

The dial comes around

0/5 complete

An angle becomes a position

Ada finds a brass dial on an old field box. Turning it changes the pointer's position, but a full turn brings the pointer home. She needs a way to compose turns without losing that circular behavior.

Mathlib's `Circle` is the unit circle in the complex plane. The map `Circle.exp` sends a real angle to a point on that circle. Mathlib also knows that the circle is an analytic Lie group, so composing positions and moving smoothly are part of the same structure.

01	No turn leaves the pointer home	Local kernel
02	Two turns compose	Local kernel
03	One full turn changes nothing	Local kernel
04	Two angles, one pointer position	Local kernel
05	The pointer turns smoothly	Local kernel

Convenience port of Lean4Game; upstream creators and licenses retain credit.

Mathlib proof checking stays in this browser.

An angle becomes a position

Ada finds a brass dial on an old field box. Turning it changes the pointer's position, but a full turn brings the pointer home. She needs a way to compose turns without losing that circular behavior.

Mathlib's `Circle` is the unit circle in the complex plane. The map `Circle.exp` sends a real angle to a point on that circle. Mathlib also knows that the circle is an analytic Lie group, so composing positions and moving smoothly are part of the same structure.

Measured: world page opened in 25 ms.

The Manifold Adventure r5 · page 58 of 75

The dial comes around · level 1 of 5 · `circlemotion-1`

33. No turn leaves the pointer home

The screenshot shows the Lean theorem prover interface. The main window displays the goal: `!- Circle.exp 0 = 1`. The left sidebar contains a list of level rewards, including `Circle.exp` and `Circle.exp_zero`. The right sidebar shows a list of available tactics, including `apply`, `by_cases`, `calc`, `constructor`, `exact`, `ext`, `fun_ext`, `have`, `infer_instance`, `intro`, `left`, `obtain`, `rcases`, `refl`, `right`, `rw`, `simp`, `simplify`, and `unfold`. The bottom of the interface shows buttons for `Verify answer`, `Show a hint`, and `View solution`.

Ada starts with the pointer at its home mark. Before she turns the dial, its angle is zero and its position on the circle is one.

The identity of the circle group is `1`. Mathlib records the zero-angle calculation as `Circle.exp_zero`.

Objective: Show that angle zero gives the identity position on the circle.

GOAL

```
theorem circle_zero_turn : Circle.exp 0 = 1
```

SOLUTION

```
exact Circle.exp_zero
```

WHY IT WORKS

The zero-angle value is a recorded calculation. Use `Circle.exp_zero`.

The dial's home position now agrees with the circle-group identity.

Unlocks: theorems `Circle.exp_zero`

Measured: navigation 27 ms, Lean ready 1 ms, goal preview 794 ms, verify 63 ms (kernel 18 ms), accepted by the kernel.

The dial comes around · level 2 of 5 · `circlemotion-2`

34. Two turns compose

The screenshot shows the Lean 4 IDE interface for a problem titled "Two turns compose". The interface is divided into several panels:

- Left Panel:** Contains the problem description and a "Level rewards" table.
- Top Panel:** Shows the "Objective: Prove that adding two angles agrees with composing their circle positions." and the "Initial goal" statement.
- Center Panel:** Displays the "Your proof" area with a "by" prompt and a "1" indicating the first step.
- Right Panel:** Lists "Available tactics" such as `apply`, `by_cases`, `calc`, `constructor`, `exact`, `ext`, `fun_undef`, `have`, `infer_instance`, `intro`, `left`, `obtain`, `rcases`, `rfl`, `right`, `rw`, `simp`, and `unfolding`.
- Bottom Panel:** Includes buttons for "Verify answer", "Show a hint", and "View solution".

The "Level rewards" table lists various tasks and their completion status:

Task	Status
One ball, one leaf	4/4
One pole is missing	5/5
A stack of maps	5/5
Smooth manifolds	5/5
A direction at every point	5/5
Identify and product charts	5/5
How smooth is smooth	5/5

Ada turns the dial once, then turns it again. The final position is the same as adding the two angles before moving the pointer.

Circle positions compose by multiplication. The theorem `Circle.exp_add` says that `Circle.exp` changes addition of angles into multiplication on the circle. This is the group law used for planar rotations.

Objective: Prove that adding two angles agrees with composing their circle positions.

GOAL

```
theorem circle_turns_compose (first second : ℝ) :  
  Circle.exp (first + second) = Circle.exp first * Circle.exp second
```

SOLUTION

```
exact Circle.exp_add first second
```

WHY IT WORKS

The exponential turns angle addition into circle multiplication. Give both angles to `Circle.exp_add`.

Ada can combine consecutive turns with the circle-group operation.

Unlocks: theorems `Circle.exp_add`

Measured: navigation 33 ms, Lean ready 2 ms, goal preview 792 ms, verify 105 ms (kernel 28 ms), accepted by the kernel.

circlemotion-3

35. One full turn changes nothing

[illegible]

Ada turns the dial through one complete revolution. The pointer travels, but it finishes at the position where it began.

Angles on the real line are not unique coordinates for a circle point. Adding `2 * Real.pi` gives the same point. Mathlib names this calculation `Circle.exp_add_two_pi`.

Objective: Show that adding one full turn does not change the pointer's position.

GOAL

```
theorem circle_full_turn (angle : ℝ) :
  Circle.exp (angle + 2 * Real.pi) = Circle.exp angle
```

SOLUTION

```
exact Circle.exp_add_two_pi angle
```

WHY IT WORKS

Mathlib records what adding one revolution does. Use `Circle.exp_add_two_pi angle`.

The formal dial now returns to the same state after one revolution.

Unlocks: theorems `Circle.exp_add_two_pi`

Measured: navigation 37 ms, Lean ready 1 ms, goal preview 790 ms, verify 67 ms (kernel 21 ms), accepted by the kernel.

36. Two angles, one pointer position

The screenshot shows the Lean theorem prover interface. The main window displays a proof goal: `! Z, first = second + turns * (2 * Real.pi)`. The left sidebar contains a table of progress for various levels, with 'Two angles, one pointer position' highlighted. The right sidebar shows available tactics like `apply`, `by_cases`, `calc`, `constructor`, etc.

Ada records the dial with two different angle readings and sees the pointer in the same place. The only way that can happen is that the two readings differ by some whole number of full turns.

This is the exact rule for when two angle coordinates name the same circle point, and it is why an angle is a local chart rather than a global one. Mathlib states it as `Circle.exp_eq_exp`: the positions agree if and only if the angles differ by an integer multiple of $2 * \text{Real.pi}$. The integer is coerced to a real number in the statement.

Objective: Show that two angles give the same pointer position exactly when they differ by a whole number of turns.

Think first: Before proving it: how long can an arc of the dial be before one angle reading stops being a chart of it?

Answer: Strictly less than a full turn. On an arc of length 2π or more, two different angles would name the same point, so the reading is no longer one-to-one.

GOAL

```
theorem same_point_iff_full_turns (first second : ℝ) :
  Circle.exp first = Circle.exp second ↔
    ∃ turns : ℤ, first = second + turns * (2 * Real.pi)
```

SOLUTION

```
exact Circle.exp_eq_exp
```

WHY IT WORKS

Angles that agree on the circle differ by whole turns. The two-way statement is `Circle.exp_eq_exp`.

An angle chart of the dial is honest on any arc shorter than a full turn, and on the whole circle it repeats.

Unlocks: theorems `Circle.exp_eq_exp`

Measured: navigation 38 ms, Lean ready 2 ms, goal preview 793 ms, verify 80 ms (kernel 26 ms), accepted by the kernel.

37. The pointer turns smoothly

The screenshot shows the Lean 4 IDE interface for a level titled "The pointer turns smoothly". The main area displays the objective: "Prove that converting an angle into a circle position is smooth." and the initial goal: $\vdash \text{CMDiff} \approx \text{Circle.exp}$. The "Your proof" section shows a single tactic `by 1`. The right sidebar lists available tactics such as `apply`, `by_cases`, `calc`, `constructor`, `exact`, `ext`, `fun_ext`, `have`, `infer_instance`, `intro`, `left`, `obtain`, `rcases`, `refl`, `right`, `rw`, `simp`, `simp`, and `unfold`. The left sidebar shows level rewards and a world tree.

Ada turns the dial slowly. The pointer follows without a jump or corner, even when it crosses the home mark.

The statement `CMDiff  $\approx$  Circle.exp` uses Mathlib's manifold notation. `CMDiff n f` elaborates to `ContMDiff I J n f`, with the two models inferred instead of written out. It says that the angle-to-circle map has derivatives of every finite order as a map between manifolds. Mathlib proves this in `contMDiff_circleExp`.

Objective: Prove that converting an angle into a circle position is smooth.

GOAL

```
theorem circle_turning_is_smooth : CMDiff  $\approx$  Circle.exp
```

SOLUTION

```
exact contMDiff_circleExp
```

WHY IT WORKS

Mathlib already knows the circle exponential is manifold-smooth. Use `contMDiff_circleExp`.

A continuously turning angle now gives smooth motion on the circle.

Unlocks: theorems `contMDiff_circleExp`

Measured: navigation 42 ms, Lean ready 4 ms, goal preview 4.3 s, verify 112 ms (kernel 51 ms), accepted by the kernel.

Two hinges, one reach

Manifold Adventure

WIP Work in progress

Games

Worlds

Lean playground

Game source

Back to world tree

LOCAL KERNEL

Two hinges, one reach

0/4 complete

Where the arm can reach

Inside the field box, Ada finds a small arm with two rotating hinges. Each hinge position lies on Mathlib's `Circle`, and reading both rings at once gives one point of `Circle × Circle`. This is the arm's configuration space, a concrete torus and the product manifold of the main path. A value of a product type is written with plain parentheses, as in `(shoulder, elbow)`.

We represent the work surface by `ℂ`, viewed as a plane. The first link points in the shoulder direction. The second link turns by the shoulder and elbow angles together.

01 Find the tip of the armLocal kernel

02 Both bars point forwardLocal kernel

03 A full shoulder turn reaches the same pointLocal kernel

04 The arm moves without a jumpLocal kernel

Where the arm can reach

Inside the field box, Ada finds a small arm with two rotating hinges. Each hinge position lies on Mathlib's `Circle`, and reading both rings at once gives one point of `Circle × Circle`. This is the arm's configuration space, a concrete torus and the product manifold of the main path. A value of a product type is written with plain parentheses, as in `(shoulder, elbow)`.

We represent the work surface by `ℂ`, viewed as a plane. The first link points in the shoulder direction. The second link turns by the shoulder and elbow angles together.

Measured: world page opened in 323 ms.

The Manifold Adventure r5 · page 64 of 75

38. Find the tip of the arm

Two hinges, one reach · Level 1 of 4

Find the tip of the arm

Ada follows the first bar from the base, then the second bar from the elbow. Adding those two displacements gives the tip position.

Complex numbers describe vectors in the work plane. The first displacement uses `joints.1`. The second multiplies two `Circle` values as `joints.1 * joints.2`, because the elbow direction is measured after the shoulder has already turned.

This is a definition level, so any well-typed term would satisfy the kernel. The next three levels reason about the course's official `robot_arm_tip`, so match the two-link formula described here.

Objective: Define the tip as the sum of the two link vectors.

Active goal: Initial goal

Objects: `firstLength secondLength : ℝ`
`joints : Circle × Circle`

Your proof: Enter text only. The surrounding `by` is supplied for you.

```
by 1
```

Available tactics: `apply` `by_cases` `calc` `constructor` `exact` `ext` `fun_undef` `have` `infer_instance` `intro` `left` `obtain` `rcases` `rfl` `right` `rw` `simp` `simp` `unfold`

World tree:

World tree	Progress
One trail, one leaf	4/4
One pole is missing	5/5
A stack of maps	5/5
Smooth manifolds	5/5
A direction at every point	5/5
Identify and product charts	5/5
How smooth is smooth	3/3

Local Lean kernel ready

Ada follows the first bar from the base, then the second bar from the elbow. Adding those two displacements gives the tip position.

Complex numbers describe vectors in the work plane. The first displacement uses `joints.1`. The second multiplies two `Circle` values as `joints.1 * joints.2`, because the elbow direction is measured after the shoulder has already turned.

This is a definition level, so any well-typed term would satisfy the kernel. The next three levels reason about the course's official `robot_arm_tip`, so match the two-link formula described here.

Objective: Define the tip as the sum of the two link vectors.

GOAL

```
noncomputable def robot_arm_tip (firstLength secondLength : ℝ)
  (joints : Circle × Circle) : ℂ
```

SOLUTION

```
exact
  (firstLength : ℂ) * (joints.1 : ℂ) +
    (secondLength : ℂ) * ((joints.1 * joints.2 : Circle) : ℂ)
```

WHY IT WORKS

The first link contributes `firstLength * joints.1`. The second direction is the product `joints.1 * joints.2`.

The configuration now determines a point on the work surface.

Measured: navigation 142 ms, Lean ready 208 ms, goal preview 834 ms, verify 202 ms (kernel 46 ms), accepted by the kernel.

39. Both bars point forward

The screenshot shows the Lean theorem prover interface. The main window displays the following content:

Objective: Compute the tip position when both joints are at the identity.

Active goal:

```

theorem robot_arm_at_rest (firstLength secondLength : ℝ) :
  robot_arm_tip firstLength secondLength (1, 1) =
    (firstLength + secondLength : ℝ)

```

Objects:

```

firstLength secondLength : ℝ

```

Your proof:

```

by
  1

```

The interface also includes a sidebar with a list of problems, a 'Level rewards' section, and a 'Verification scope' section.

Ada returns both hinges to their home marks. The two bars lie in one straight line, so the tip sits at the sum of their lengths.

The identity `1` in `Circle` points along the positive real axis. Unfolding `robot_arm_tip` leaves a direct complex-number calculation. The `simp` tactic knows the identity laws involved.

Objective: Compute the tip position when both joints are at the identity.

GOAL

```

theorem robot_arm_at_rest (firstLength secondLength : ℝ) :
  robot_arm_tip firstLength secondLength (1, 1) =
    (firstLength + secondLength : ℝ)

```

SOLUTION

```

simp [robot_arm_tip]

```

WHY IT WORKS

This is a direct computation with the definition unfolded. Simplify with `simp [robot_arm_tip]`.

At rest, the arm reaches straight ahead by the sum of its link lengths.

Measured: navigation 91 ms, Lean ready 1 ms, goal preview 792 ms, verify 111 ms (kernel 39 ms), accepted by the kernel.

40. A full shoulder turn reaches the same point

Two hinges, one reach · Level 3 of 4 · robotarm-3

Objective: Show that adding a full turn to the shoulder angle leaves the endpoint unchanged.

Active goal

```

theorem robot_full_turn_same_tip (firstLength secondLength shoulder elbow : ℝ) :
  robot_arm_tip firstLength secondLength
    (Circle.exp (shoulder + 2 * Real.pi), Circle.exp elbow) =
  robot_arm_tip firstLength secondLength
    (Circle.exp shoulder, Circle.exp elbow)
  
```

Objects

```

firstLength secondLength : ℝ
shoulder elbow
  
```

Initial goal

```

robot_arm_tip firstLength
secondLength
  (Circle.exp (shoulder +
2 * Real.pi), Circle.exp elbow)
  
```

Your proof

Enter tactics only. The surrounding `by` is supplied for you. 0 attempts

```

by 1
  
```

Available tactics

apply | by_cases | calc | constructor
 exact | ext | fun_arg | have
 infer_instance | intro | left | obtain
 rcases | rfl | right | rw | simp
 skip | unfold | ...

► Verification scope

World tree

- One trail, one leaf 4/4
- One pole is missing 5/5
- A stack of maps 5/5
- Smooth manifolds 5/5
- A direction at every point 5/5
- Identity and product charts 5/5
- How smooth is smooth 5/5

Ada rotates the shoulder through one complete turn while leaving the elbow reading alone. The arm sweeps around and returns to the same physical pose.

The previous world proved `Circle.exp_add_two_pi`. Rewriting that one joint makes the two configurations, and therefore their tip positions, equal.

Objective: Show that adding a full turn to the shoulder angle leaves the endpoint unchanged.

GOAL

```

theorem robot_full_turn_same_tip (firstLength secondLength shoulder elbow : ℝ) :
  robot_arm_tip firstLength secondLength
    (Circle.exp (shoulder + 2 * Real.pi), Circle.exp elbow) =
  robot_arm_tip firstLength secondLength
    (Circle.exp shoulder, Circle.exp elbow)
  
```

SOLUTION

```

rw [Circle.exp_add_two_pi]
  
```

WHY IT WORKS

Only the shoulder reading differs between the two configurations. Rewrite it with `Circle.exp_add_two_pi`.

Different real angles can now describe the same arm pose.

Measured: navigation 66 ms, Lean ready 1 ms, goal preview 825 ms, verify 100 ms (kernel 24 ms), accepted by the kernel.

41. The arm moves without a jump

The screenshot shows the Manifold Adventure game interface. The title bar indicates 'Two hinges, one reach · Level 4 of 4' and 'The arm moves without a jump'. The left sidebar contains a 'World tree' with items like 'One ball, one leaf', 'One pole is missing', 'A stack of maps', 'Smooth manifolds', 'A direction at every point', 'Identify and product charts', and 'How smooth is smooth'. The main content area displays the objective: 'Prove that the forward-kinematics map from joint states to tip positions is continuous.' It also shows an 'Active goal' and 'Objects' section. The 'Your proof' section has a text input area with 'by 1' and buttons for 'Verify answer', 'Show a hint', and 'View solution'. The right sidebar lists 'Available tactics' such as 'apply', 'by_cases', 'calc', 'constructor', 'exact', 'ext', 'fun_prop', 'have', 'infer_instance', 'intro', 'left', 'obtain', 'rcases', 'rfl', 'right', 'rw', 'simp', 'skip', 'unfold', and 'Verification scope'.

Ada nudges either hinge. The tip moves with it instead of jumping to a distant point on the table.

The coordinate projections from $\text{Circle} \times \text{Circle}$ are `Continuous`. Coercing a circle point into $\mathbb{C}$ is continuous too, and sums and products of continuous complex-valued functions stay continuous. The proof assembles those facts in the same order as the arm formula.

Objective: Prove that the forward-kinematics map from joint states to tip positions is continuous.

GOAL

```
theorem robot_arm_tip_continuous (firstLength secondLength : ℝ) :
  Continuous (robot_arm_tip firstLength secondLength)
```

SOLUTION

```
unfold robot_arm_tip
exact (continuous_const.mul (continuous_subtype_val.comp continuous_fst)).add
      (continuous_const.mul ((continuous_subtype_val.comp continuous_fst).mul
                             (continuous_subtype_val.comp continuous_snd)))
```

WHY IT WORKS

Unfold `robot_arm_tip` so that the two link contributions are visible. Build continuity with `.comp`, `.mul`, and `.add` in the same shape as the formula.

Small changes at the hinges now produce small changes at the tip. Now that Ada has built the proof by hand, the course gives her the power tool: with `fun_prop` unlocked, `unfold robot_arm_tip; fun_prop` closes the same goal in one line.

Unlocks: tactics `unfold`; on completion `fun_prop`; theorems `continuous_const`, `continuous_subtype_val`, `continuous_fst`, `continuous_snd`, `Continuous.comp`, `Continuous.mul`, `Continuous.add`

Measured: navigation 255 ms, Lean ready 22 ms, goal preview 809 ms, verify 152 ms (kernel 67 ms), accepted by the kernel.

Can the arm touch it?

Manifold Adventure

WIP · Work in progress

Games

Worlds

Lean playground

Game source

Back to world tree

LOCAL KERNEL

Can the arm touch it?

0/4 complete

The ring of reach

Ada sees a crumb on the work surface and asks a practical question before turning either hinge: can the tip touch it at all? The two bars can stretch only so far, and when one is longer, folding the shorter bar leaves a gap near the base.

For nonnegative lengths `firstLength` and `secondLength`, every endpoint lies between the radii $|\text{firstLength} - \text{secondLength}|$ and $\text{firstLength} + \text{secondLength}$. Each link direction is a point of Mathlib's `Circle`, while the endpoint lies in the complex plane. The interactive lab turns those inequalities into a shaded annulus. Drag the target to see the two inverse-kinematics poses meet at its boundaries.

The three-link switch is an outlook. An extra hinge can close the central gap and turns isolated solutions into a continuous family. The Lean levels keep their formal argument on the two-link arm, where the obstruction is already useful and precise.

Reachability lab

Two-link arm

Two links

Three links

First link

2.0

Second link

1.0

Target radius

2.20

Target angle

22°

01

The arm has an outer limit

Local kernel

02

The folded arm leaves a gap

Local kernel

03

Every pose stays in the ring

Local kernel

04

Outside the ring is out of reach

Local kernel

The ring of reach

Ada sees a crumb on the work surface and asks a practical question before turning either hinge: can the tip touch it at all? The two bars can stretch only so far, and when one is longer, folding the shorter bar leaves a gap near the base.

For nonnegative lengths `firstLength` and `secondLength`, every endpoint lies between the radii $|\text{firstLength} - \text{secondLength}|$ and $\text{firstLength} + \text{secondLength}$. Each link direction is a point of Mathlib's `Circle`, while the endpoint lies in the complex plane. The interactive lab turns those inequalities into a shaded annulus. Drag the target to see the two inverse-kinematics poses meet at its boundaries.

The three-link switch is an outlook. An extra hinge can close the central gap and turns isolated solutions into a continuous family. The Lean levels keep their formal argument on the two-link arm, where the obstruction is already useful and precise.

Measured: world page opened in 25 ms.

The Manifold Adventure r5 · page 69 of 75

42. The arm has an outer limit

Can the arm touch it? / Level 1 of 4

The arm has an outer limit

Ada straightens both bars toward the crumb. Even in this longest pose, the tip cannot travel farther than the two bar lengths added together.

The endpoint is a sum of two complex displacement vectors. The triangle inequality `norm_add_le` bounds the norm of their sum by the sum of their norms. Each `Circle` direction has norm one, recorded by `Circle.norm_coe`, so the two terms simplify to the two nonnegative lengths.

Objective: Prove that the endpoint is no farther from the base than the sum of the link lengths.

Active goal

Objects

```
firstLength secondLength : ℝ
joints : Circle × Circle
```

Assumptions

```
hFirst : 0 ≤ firstLength
hSecond : 0 ≤ secondLength
```

Your proof

Enter tactics only. The surrounding `by` is supplied for you. 0 attempts

by 1

Available tactics

reply by_cases calc constructor
exact ext fun_arg have
infer_instance intro left obtain
rcases rfl right rw simp
simp unfold

Verification scope

World tree

One ball, one leaf	4/4
One pole is missing	5/5
A stack of maps	5/5
Smooth manifolds	5/5
A direction at every point	5/5
Identify and product charts	5/5
How smooth is smooth	5/5

Verify answer Show a hint View solution

Ada straightens both bars toward the crumb. Even in this longest pose, the tip cannot travel farther than the two bar lengths added together.

The endpoint is a sum of two complex displacement vectors. The triangle inequality `norm_add_le` bounds the norm of their sum by the sum of their norms. Each `Circle` direction has norm one, recorded by `Circle.norm_coe`, so the two terms simplify to the two nonnegative lengths.

Objective: Prove that the endpoint is no farther from the base than the sum of the link lengths.

GOAL

```
theorem robot_tip_norm_le (firstLength secondLength : ℝ)
  (hFirst : 0 ≤ firstLength) (hSecond : 0 ≤ secondLength)
  (joints : Circle × Circle) :
  ||robot_arm_tip firstLength secondLength joints|| ≤
  firstLength + secondLength
```

SOLUTION

```
unfold robot_arm_tip
calc
  _ ≤ ||(firstLength : ℝ) * (joints.1 : ℝ)|| +
    ||(secondLength : ℝ) * ((joints.1 * joints.2 : Circle) : ℝ)|| := norm_add_le _ _
  _ = firstLength + secondLength := by
    simp [Circle.norm_coe, abs_of_nonneg, hFirst, hSecond]
```

WHY IT WORKS

Treat the endpoint as the sum of the two link vectors, then bound the length of that sum. Unfold `robot_arm_tip`, use a `calc` block with `norm_add_le`, then simplify the two unit-circle norms.

The outer dashed circle is now a proved limit, not just a feature of the drawing.

Unlocks: tactics `calc`; theorems `norm_add_le`, `Circle.norm_coe`

Measured: navigation 37 ms, Lean ready 2 ms, goal preview 1.3 s, verify 219 ms (kernel 78 ms), accepted by the kernel.

43. The folded arm leaves a gap

The screenshot shows the Manifold Adventure game interface. The title bar indicates 'Can the arm touch it? · Level 2 of 4 · The folded arm leaves a gap'. The sidebar on the left shows a progress list with items like 'One ball, one leaf', 'One pole is missing', 'A stack of maps', 'Smooth manifolds', 'A direction at every point', 'Identify and product charts', and 'How smooth is smooth'. The main workspace contains a diagram of a two-link arm with joints and links. The right-hand panel shows a list of available tactics (reply, by_cases, calc, constructor, exact, ext, fun_arg, have, infer_instance, intro, left, obtain, rcases, rfl, right, rw, simp, simp, unfold) and a verification scope section.

Ada folds the second bar back toward the base. If one bar is longer, the shorter one cannot cancel all of it, so a circular gap remains around the hinge.

The reverse triangle inequality appears in Mathlib as `norm_sub_norm_le`. Since absolute value asks for both orders of subtraction, `abs_sub_le_iff` splits the claim into `firstLength - secondLength ≤ ...` and its mirror image. Each half uses the same reverse-triangle argument with the bars exchanged.

Objective: Prove that the endpoint stays at least the difference of the link lengths away from the base.

GOAL

```
theorem robot_tip_norm_ge (firstLength secondLength : ℝ)
  (hFirst : 0 ≤ firstLength) (hSecond : 0 ≤ secondLength)
  (joints : Circle × Circle) :
  |firstLength - secondLength| ≤
  ||robot_arm_tip firstLength secondLength joints||
```

SOLUTION

```
unfold robot_arm_tip
apply abs_sub_le_iff.mpr
constructor
· have h := norm_sub_norm_le
  ((firstLength : ℝ) * (joints.1 : ℝ))
  (-(secondLength : ℝ) * ((joints.1 * joints.2 : Circle) : ℝ))
  simpa [Circle.norm_coe, abs_of_nonneg, hFirst, hSecond] using h
· have h := norm_sub_norm_le
  ((secondLength : ℝ) * ((joints.1 * joints.2 : Circle) : ℝ))
  (-(firstLength : ℝ) * (joints.1 : ℝ))
  simpa [Circle.norm_coe, abs_of_nonneg, hFirst, hSecond, add_comm] using h
```

WHY IT WORKS

Absolute value hides two inequalities. Prove the reverse-triangle estimate in both orders. Use `abs_sub_le_iff.mpr`, split with `constructor`, then apply `norm_sub_norm_le` to one link and the negative of the other.

The hole around the base now has the exact lower radius forced by the two bars.

Unlocks: tactics `have`, `simp`; theorems `norm_sub_norm_le`, `abs_sub_le_iff`

Measured: navigation 32 ms, Lean ready 2 ms, goal preview 1.3 s, verify 316 ms (kernel 187 ms), accepted by the kernel.

44. Every pose stays in the ring

The screenshot shows the Lean theorem prover interface. The main window displays a proof goal: "Every pose stays in the ring". The goal is to prove that the robot arm tip stays within a closed annulus defined by two concentric circles. The interface includes a sidebar with a table of progress, a main area for the proof goal, and a right-hand panel with available tactics and a verification scope.

Progress	Score
One ball, one leaf	4/4
One pole is missing	5/5
A stack of maps	5/5
Smooth manifolds	5/5
A direction at every point	5/5
Identify and product charts	5/5
How smooth is smooth	5/5

Objective: Combine the two radius bounds to show that every endpoint lies in the closed annulus.

Active goal:

```

(hFirst : 0 ≤ firstLength) (hSecond : 0 ≤ secondLength)
(joints : Circle × Circle) :
|firstLength - secondLength| ≤
  ||robot_arm_tip firstLength secondLength joints|| ∧
  ||robot_arm_tip firstLength secondLength joints|| ≤
    firstLength + secondLength

```

Available tactics: apply, by_cases, calc, constructor, exact, ext, fun_args, have, infer_instance, intro, left, obtain, rcases, rfl, right, rw, simp, simp, unfold, ...

Ada lays the two limits over the work surface. Every pose of the arm must land in the ring between them.

The goal is the conjunction of the lower and upper bounds from the previous levels. Each endpoint still comes from two `Circle`-valued joints. This is where the two inequalities become one reusable description of the arm's workspace obstruction. Split the conjunction and cite the course declarations you have just proved.

Objective: Combine the two radius bounds to show that every endpoint lies in the closed annulus.

GOAL

```

theorem robot_tip_mem_reach_annulus (firstLength secondLength : ℝ)
  (hFirst : 0 ≤ firstLength) (hSecond : 0 ≤ secondLength)
  (joints : Circle × Circle) :
  |firstLength - secondLength| ≤
    ||robot_arm_tip firstLength secondLength joints|| ∧
    ||robot_arm_tip firstLength secondLength joints|| ≤
      firstLength + secondLength

```

SOLUTION

```

constructor
· exact robot_tip_norm_ge firstLength secondLength hFirst hSecond joints
· exact robot_tip_norm_le firstLength secondLength hFirst hSecond joints

```

WHY IT WORKS

The two halves of this conjunction are exactly the previous two course declarations. Use `constructor`, then apply `robot_tip_norm_ge` and `robot_tip_norm_le`.

Every configuration on the torus now maps into the shaded ring.

Measured: navigation 31 ms, Lean ready 1 ms, goal preview 793 ms, verify 71 ms (kernel 22 ms), accepted by the kernel.

45. Outside the ring is out of reach

Can the arm touch it? / Level 4 of 4

Outside the ring is out of reach

Ada places the crumb outside the shaded ring. No amount of turning can make the tip land there: either the crumb is inside the folded gap or it lies beyond both bars.

Reachability is written as an existential statement: some $\text{joints} : \text{Circle} \times \text{Circle}$ send the tip to point , with Circle carrying each joint angle modulo a full turn. Assume such joints exist, replace point with their endpoint, then split the two ways outside can hold. Each branch contradicts one of the bounds already proved. The lab constructs poses for interior targets; this level proves the complementary obstruction in Lean.

Objective: Prove that a target outside the annulus has no inverse-kinematics solution.

Initial goal

Objects

```
firstLength secondLength : ℝ
point : ℂ
```

Assumptions

```
hFirst : 0 ≤ firstLength
hSecond : 0 ≤ secondLength
outside : [point] < [firstLength
  ~ secondLength] ∨
  [firstLength +
    secondLength] < [point]
```

Circle × Circle, robot_arm_tip firstLength secondLength joints = point

Your proof

Enter tactics only. The surrounding by is supplied for you.

by 1

Available tactics

apply | by_cases | calc | constructor

exact | ext | fun_arg | have

infer_instance | intro | left | obtain

rcases | rfl | right | rw | simp

skip | unfold

Verification scope

World tree

One trail, one leaf	4/4
One pole is missing	5/5
A stack of maps	5/5
Smooth manifolds	5/5
A direction at every point	5/5
Identity and product charts	5/5
How smooth is smooth	5/5

Ada places the crumb outside the shaded ring. No amount of turning can make the tip land there: either the crumb is inside the folded gap or it lies beyond both bars.

Reachability is written as an existential statement: some $\text{joints} : \text{Circle} \times \text{Circle}$ send the tip to point , with Circle carrying each joint angle modulo a full turn. Assume such joints exist, replace point with their endpoint, then split the two ways outside can hold. Each branch contradicts one of the bounds already proved. The lab constructs poses for interior targets; this level proves the complementary obstruction in Lean.

Objective: Prove that a target outside the annulus has no inverse-kinematics solution.

GOAL

```
theorem robot_target_outside_annulus_unreachable (firstLength secondLength : ℝ)
  (hFirst : 0 ≤ firstLength) (hSecond : 0 ≤ secondLength)
  (point : ℂ)
  (outside : ||point|| < |firstLength - secondLength| ∨
    firstLength + secondLength < ||point||) :
  ¬ ∃ joints : Circle × Circle,
    robot_arm_tip firstLength secondLength joints = point
```

SOLUTION

```
intro reaches
obtain {joints, rfl} := reaches
rcases outside with tooClose | tooFar
· exact (not_lt_of_ge
  (robot_tip_norm_ge firstLength secondLength hFirst hSecond joints)) tooClose
· exact (not_lt_of_ge
  (robot_tip_norm_le firstLength secondLength hFirst hSecond joints)) tooFar
```

WHY IT WORKS

Assume a reaching configuration exists, substitute its endpoint for the target, then contradict the appropriate radius bound. Use `intro`, unpack the existential with `obtain`, split `outside` with `rcases`, and close each branch with `not_lt_of_ge`.

Ada can now reject an impossible target before moving either hinge.

Unlocks: tactics `obtain`, `rcases`; theorems `not_lt_of_ge`

Measured: navigation 40 ms, Lean ready 9 ms, goal preview 795 ms, verify 111 ms (kernel 29 ms), accepted by the kernel.